

Sebastian Vintoñuke – 106063

Cindy Teresa Hsieh – 108051

Maria Delfina Cano Ros Langrehr – 109338

Lautaro Martin Sotelo – 107472

Toda afirmación que se realice debe estar apoyada en **evidencia**. Si se encuentra alguna **afirmación** que no está correctamente **fundamentada** con **evidencia**, se disminuirá la nota. Esto incluye las observaciones que realicen sobre el código y su funcionamiento en general. Que no existan métricas para un determinado QA no implica que no pueda evidenciarse el efecto de una táctica.

TODO: correr un corrector ortografico

1. SISTEMA RECIBIDO

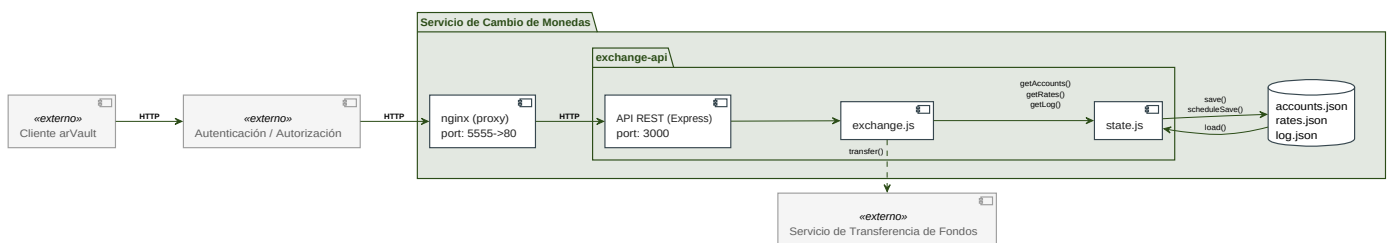


Figura 1. Diagrama Components & Connectors del sistema base.

1.1. Arquitectura

Arquitectura monolitica cliente-servidor, formada por una unica instancia app.js. Separacion en tres capas: app.js punto de entrada y definicion de endpoints, exchange.js logica de negocio, state.js persistencia. No se implementa coordinacion, no hay locks ni mecanismos de concurrencia sobre secciones criticas, como la verificacion del saldo. No se implementa tolerancia a fallos, no hay manejo de errores try/catch. No hay replicacion, unica instancia, no hay replicas. No hay recovery, si el servidor muere hay que levantarlo manualmente. No hay consistencia entre los datos en memoria y los datos en disco, la consistencia es eventual sin mecanismos de transaccionabilidad.

TODO: redactarlo mas bonito y justificar

1.2. Código

No hay separacion de responsabilidades, exchange maneja tanto logica de negocio, como logica de bajo nivel. Calcula tanto counterAmount como se encarga de persistir los cambios en counterAccount.balance.

```

1 export async function exchange(exchangeRequest) {
2   ...
3   //get the exchange rate
4   const exchangeRate = rates[baseCurrency][counterCurrency];
5   //compute the requested (counter) amount
6   const counterAmount = baseAmount * exchangeRate;
7   ...
8   //all good, update balances
9   baseAccount.balance += baseAmount;
10  counterAccount.balance -= counterAmount;
11  ...
12 }

```

Código 1. app/exchange.js

Hay alto acoplamiento, exchange.js accede directamente a variables que deberian ser privadas de state.js. exchange.js no deberia conocer detalles de implementacion de la persistencia, se deberia programar contra interfaces. Cambiar la persistencia implica modificar exchange.js. No hay modelo de clases, todo esta basado en constantes globales y funciones publicas.

```

1 let accounts;
2 let rates;
3 let log;

```

```

4
5 export async function init() {
6   await stateInit();
7
8   accounts = stateAccounts();
9   rates = stateRates();
10  log = stateLog();
11 }

```

Código 2. app/exchange.js

No se implementa transaccionalidad, posibles inconsistencias ante fallos incluso al recuperarse.

```

1 async function save(data, fileName) {
2   const filePath = path.join(__dirname, fileName);
3   try {
4     await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
5   } catch (err) {
6     console.error(`Error writing to ${filePath}:`, err);
7   }
8 }

```

Código 3. app/state.js

No se implemento un diseño guiado por dominio, se opera directamente contra los tipos de datos primitivos. Se modela el dinero utilizando floats, esto genera perdida de precision.

```

1 const exchangeRate = rates[baseCurrency][counterCurrency];
2 //compute the requested (counter) amount
3 const counterAmount = baseAmount * exchangeRate;

```

Código 4. app/exchange.js

1.3. Infraestructura

2. ATRIBUTOS DE CALIDAD

2.1. Identificación de los QA claves

Se identifican los siguientes atributos de calidad como claves en el contexto del negocio. Adicionalmente, se citan especificaciones y requerimientos que motivaron estas afirmaciones.

2.1.1. Availability (disponibilidad)

Habilidad del sistema para reparar o enmascarar fallas de manera tal que el período fuera de servicio no exceda un determinado valor durante un lapso de tiempo especificado.

La startup arVault es una fintech que opera una billetera digital

Al ser una billetera digital, el servicio permite manipular el dinero de los usuarios. Un servicio con baja o nula disponibilidad implicaría que los usuarios no pueden acceder a su propio dinero. Adicionalmente, el modelo de negocio se basa en que los usuarios operen, si el servicio no es disponible para permitir estas operaciones, se estan perdiendo potenciales ganancias.

2.1.2. Performance

Habilidad del sistema para reaccionar ante ciertos eventos en un determinado tiempo.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas.

Uno de los principales problemas que impulsaron esta review es el endpoint de /exchanges, como se mostro es (TODO) este considerablemente mas lento que el resto de endpoints, lo que podria implicar el descontento de los usuarios.

las tasas de cambio no tienen gap entre el valor de compra y de venta.

Adicionalmente, el modelo de negocios es ofrecer una tasa de cambio sin gap, es decir en tiempo real. Una performance pobre en esta operacion implicaría que las tasas podrian que tener que ajustarse negativamente en comparacion a lo que se le ofrece al usuario en primera instancia.

2.1.3. Scalability (escalabilidad)

Habilidad del sistema para incorporar recursos adicionales de manera efectiva. Efectividad = mejora mensurable de algún QA + no requerir un esfuerzo inapropiado + no interrumpir el funcionamiento.

Para ganar usuarios, las tasas de cambio...

Se especifica que objetivo a corto plazo es incrementar el número de usuarios, el cual, el sistema debe ser capaz de manejar incorporando los recursos necesarios.

2.1.4. *Reliability (confiabilidad)*

Grado en que la arquitectura es susceptible a fallas a nivel de sistema cuando se producen fallas parciales en componentes, conectores o datos.

TODO: TODO: justificar

2.1.5. *Security (seguridad)*

Capacidad del sistema de prevenir acciones maliciosas o accidentales fuera del uso esperado.

se valida que haya saldo en la cuenta del cliente, se espera que lo haga la UI

La posibilidad de que los usuarios puedan realizar acciones maliciosas implican pérdidas monetarias para el resto de usuarios, para la propia organización, además de una pérdida de reputación y con esto pérdida de usuarios e inversiones.

2.2. Otros QA menos relevantes

Además se mencionan otros QA, que aunque no resulten claves y por ende no son una prioridad, son deseables, especialmente conforme el proyecto crezca.

TODO: TODO: busca en el enunciado o en la documentación como justificar estos QA

2.2.1. *Efficiency*

Performance por recurso consumido.

2.2.2. *Testability (testeabilidad)*

Grado en que un componente/sistema de software permite ser probado para exhibir sus fallas; usualmente relacionado al esfuerzo necesario para llevar adelante las tareas de testing.

2.2.3. *Manageability*

Define la facilidad de gestión del sistema por parte de los administradores del mismo.

2.2.4. *Simplicity*

Facilidad con la que la arquitectura de un sistema puede ser entendida, razonada y explicada.

2.3. Decisiones de diseño actuales y su impacto en los QA

2.3.1. *Impacto en la Performance*

...

2.4. *Critica*

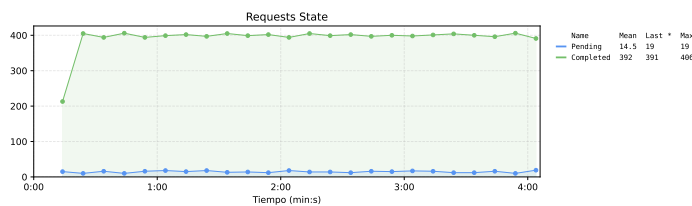
3. COMPORTAMIENTO DEL SERVICIO

3.1. Pruebas de carga

Las pruebas de carga no revelaron información significativa sobre el sistema inicial. Sin embargo aprovechamos esta sección para ilustrar el formato de los gráficos y explicar los criterios compartidos entre todos los escenarios.

Se pueden encontrar los resultados completos en el [Apendice](#).

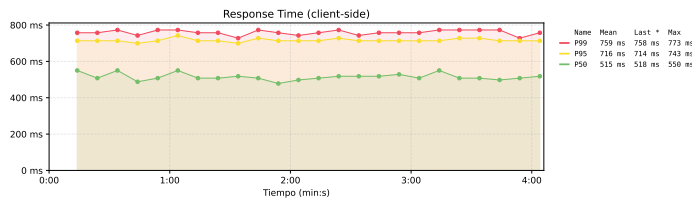
3.1.1. Disponibilidad



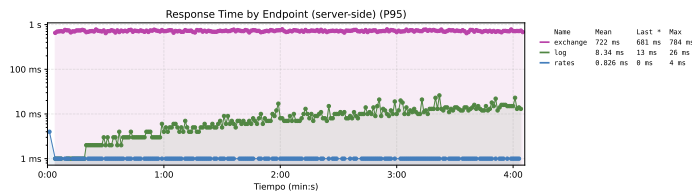
Para evaluar la disponibilidad del sistema monitoreamos el estado de las request. El eje x muestra el tiempo transcurrido de la prueba y cada punto agrupa una ventana de 10 segundos. Completed indica cuántas requests finalizaron correctamente en esa ventana, mientras que Pending indica cuántas seguían en curso. Adicionalmente en la leyenda de la derecha se muestra el promedio, el ultimo valor y el maximo.

En 4 minutos con una carga de 40 request por segundo, se puede observar que el sistema no presento inconvenientes.

3.1.2. Latencia

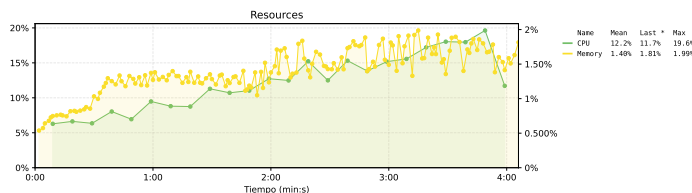


Para evaluar la latencia del sistema optamos por utilizar dos metodologías, la primera monitorear los tiempos de respuestas desde la perspectiva del cliente a través de Artillery. Optamos por realizar el monitoreo en P50, P95 Y P99, sinembargo una consideración importante para esta metodología es que no distingue entre endpoints. Se mantiene el formato de tiempo en el eje x con ventanas de 10 segundos.



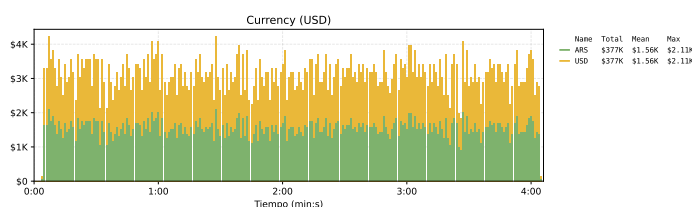
La siguiente metodología adoptada fue medir los tiempos de respuesta por endpoint desde la perspectiva del servidor. Se realizó agregando la recolección de métricas directamente en el servidor. Si bien en contraposición a la metodología anterior si podemos discernir entre endpoints, optamos por representarlo solo para P95 para no saturar la representación. Se mantiene el formato de tiempo en el eje x con ventanas de 10 segundos, sinembargo para el eje y optamos por una escala logarítmica por la gran discrepancia entre los tiempos.

3.1.3. Uso de recursos



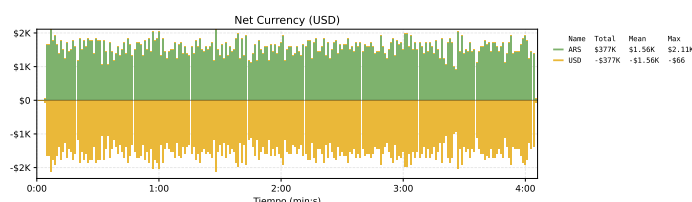
Para evaluar el uso de recursos del sistema, monitoreamos el uso del CPU y de la memoria. El uso del CPU se lee sobre el eje y al lado izquierdo mientras que el uso de la memoria al lado derecho. Nuevamente, mantiene el formato de tiempo en el eje x con ventanas de 10 segundos.

3.1.4. Volumen operado por moneda



Como la primera métrica de negocio, se evaluó el volumen operado por cada moneda. Para esto se agregó la recolección de métricas directamente en el servidor. Optamos por representar cada moneda con su respectivo valor en dólares, a la cotización vigente en cada operación, para que los valores sean comparables. Los volúmenes de diferentes monedas operados en un mismo punto se apilan para representar el total operado en ese punto. Se mantiene el formato de tiempo en el eje x pero se redujo la ventana a 1 segundo.

3.1.5. Volumen neto operado

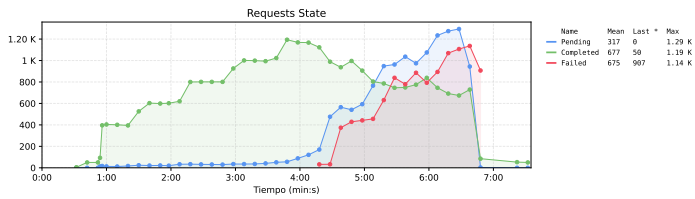


Para la segunda métrica de negocio, se evaluó el volumen neto operado por cada moneda. Nuevamente, a través de la recolección de métricas directamente en el servidor. De igual modo, con cada moneda en su respectivo valor en dólares. A diferencia del volumen operado, las ventas se representan como valores negativos y las compras con valores positivos. Se mantiene el formato de tiempo en el eje x y la ventana de 1 segundo.

3.2. Pruebas de estrés

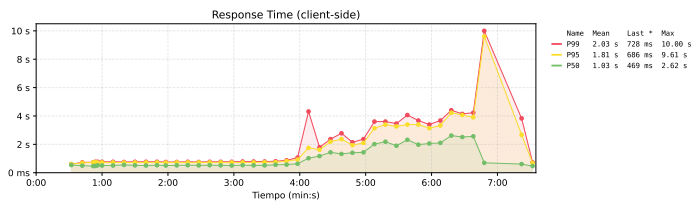
Se pueden encontrar los resultados completos en el Apéndice.

3.2.1. Disponibilidad

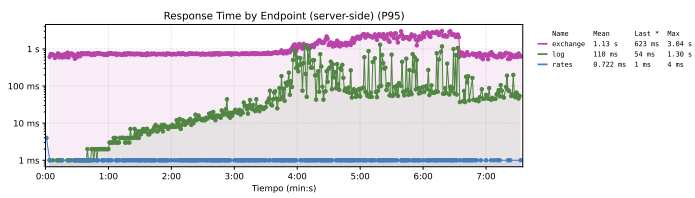


TODO: completar

3.2.2. Latencia

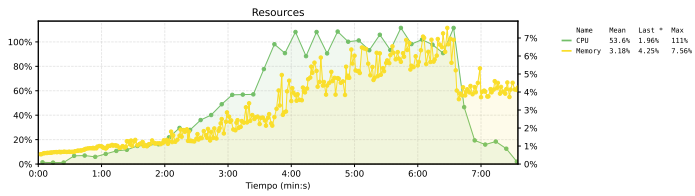


TODO: completar



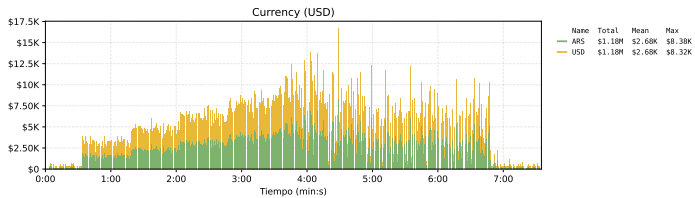
TODO: completar

3.2.3. Uso de recursos



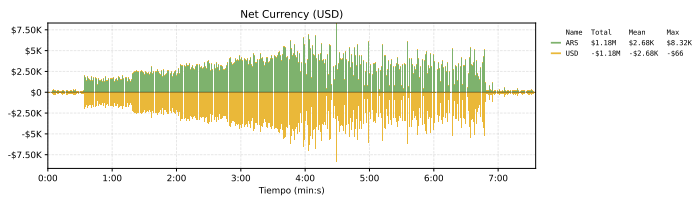
TODO: completar

3.2.4. Volumen operado por moneda



TODO: completar

3.2.5. Volumen neto operado

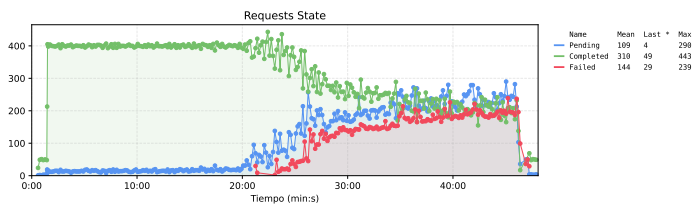


TODO: completar

3.3. Pruebas de resistencia

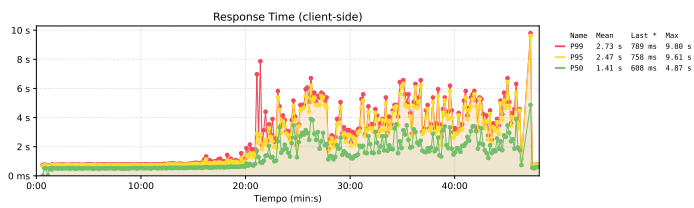
Se pueden encontrar los resultados completos en el Apendice.

3.3.1. Disponibilidad

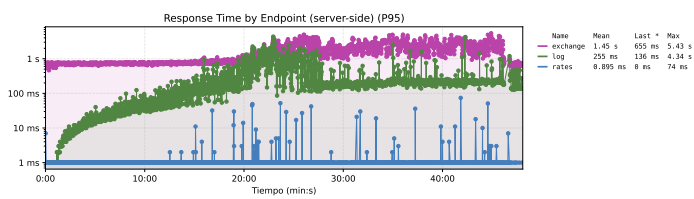


TODO: completar

3.3.2. Latencia

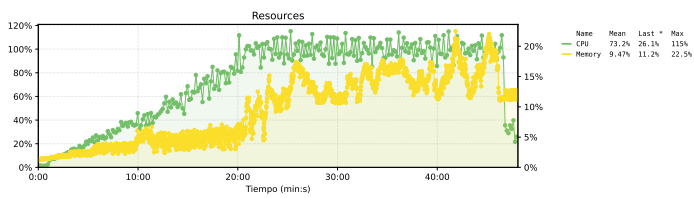


TODO: completar



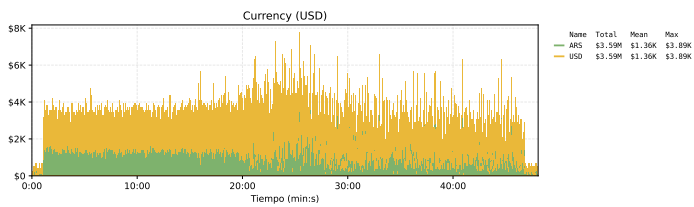
TODO: completar

3.3.3. Uso de recursos



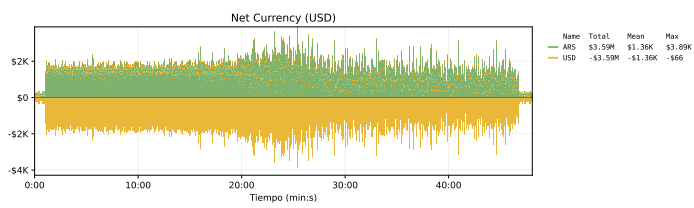
TODO: completar

3.3.4. Volumen operado por moneda



TODO: completar

3.3.5. Volumen neto operado

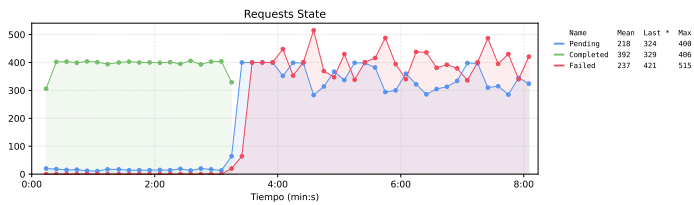


TODO: completar

3.4. Pruebas de caos

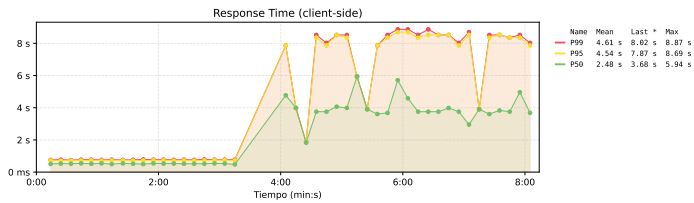
Se pueden encontrar los resultados completos en el [Apendice](#).

3.4.1. Disponibilidad

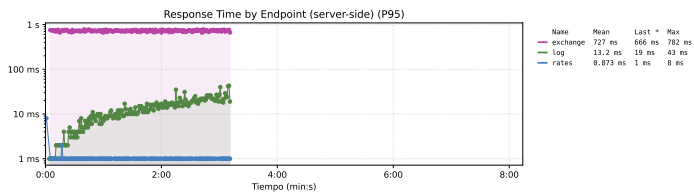


TODO: completar

3.4.2. Latencia

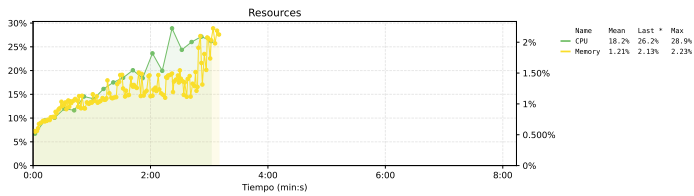


TODO: completar



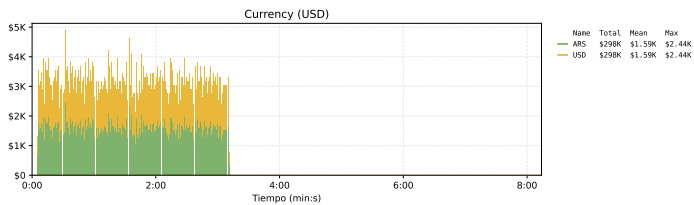
TODO: completar

3.4.3. Uso de recursos



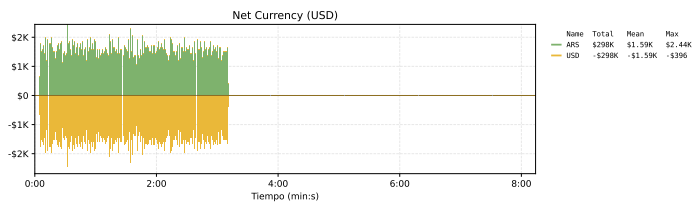
TODO: completar

3.4.4. Volumen operado por moneda



TODO: completar

3.4.5. Volumen neto operado



TODO: completar

4. PROPUESTA 1

El sistema presenta una degradación principalmente por el manejo indevido del log. Destacamos que la posibilidad de permitir consultar el log de manera completa es la principal causa de degradación. Este escala con el número de operación, lo que provoca un aumento en el uso de recursos para mantenerlo, y un aumento en los tiempos de respuesta para consultarlo. Esto arrastra la degradación a los otros endpoints, entre ellos exchange, el principalmente mencionado por los usuarios.

Destacamos que un endpoint que permite consultarlo todo por definición no es escalable, recomendamos la implementación de paginación. Sin embargo, asumimos que esta es un requerimiento de negocio y que las mejoras propuestas no deben alterar la firma del sistema para con los usuarios. Se propone desacoplar el log para que la degradación de este no afecte a los endpoints de negocio del sistema. Proponemos agregar un nodo dedicado, análogo a exchange-api para el manejo del log. Algunas optimizaciones de la implementación que consideramos que vale la pena mencionar son:

- Remplazamos el uso del formato .json para el almacenamiento a .ndjson, de esta manera podemos agregar los registros al final sin reescribir todo el archivo.
- El archivo ya no se carga en memoria, se streamea el cliente directamente desde disco en batches. Destacamos que agregamos los caracteres [al inicio y] al final del stream para mantener la firma original con respecto al formato .json.
- Solo enviamos hasta el último byte válido disponible al momento de recibida la consulta y verificamos que la última entrada sea válida (evita race condition con la escritura).
- Exchange api loguea directamente al nuevo nodo dedicado mediante una conexión TCP dedicada.

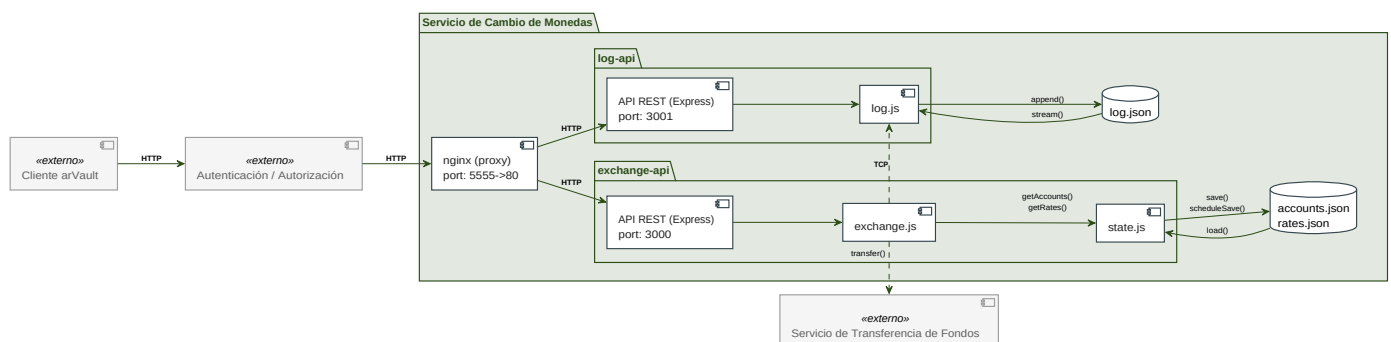


Figura 2. Diagrama Components & Connectors de la propuesta 1.

4.1. Impacto en los QA

4.1.1. Performance

TODO: evaluar el impacto en cada QA

4.2. Pruebas de carga

La prueba de carga se realizó usando una tasa constante de 40 usuarios durante 240s (4 minutos). Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests por segundo. No se realizó warm-up ni ramp para no ensuciar las métricas de promedios. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba. Se pueden encontrar los resultados completos en el [Apendice](#).

4.3. Pruebas de estrés

La prueba de estrés se realizó incrementando progresivamente la carga desde 40 hasta 180 en saltos de 20, cada salto con una duración de 45s, el tiempo total de la prueba es de 450s (7,5 minutos). Se realizó un warm-up inicial de 5 usuarios durante 30s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 60s. Tampoco se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

Se pueden encontrar los resultados completos en el [Apendice](#).

4.3.1. Disponibilidad

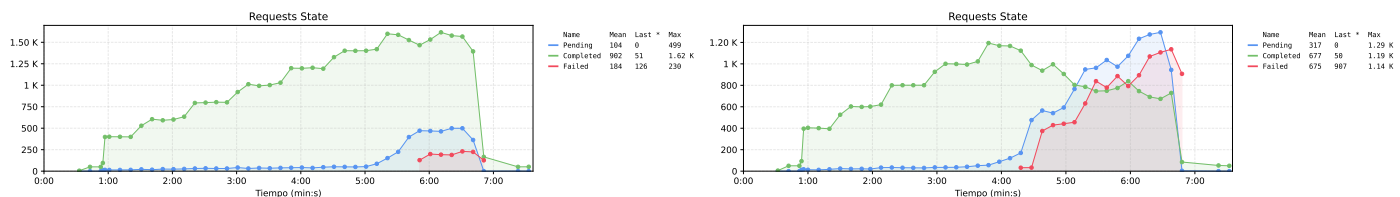


Figura 3. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. La propuesta 1 obtuvo un incremento en el promedio de respuestas satisfactorias de 225 respuestas adicionales, lo que representa un aumento del 33,23 % frente al caso base.

4.3.2. Latencia

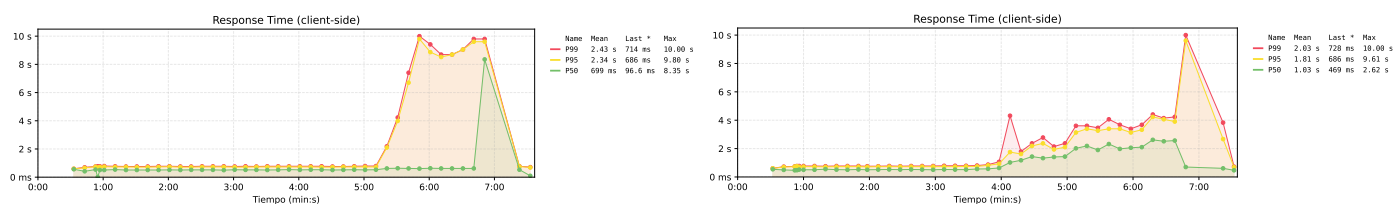


Figura 4. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 mejoró el promedio de latencia para P50. También se observa que empeoró en P95 y P99.

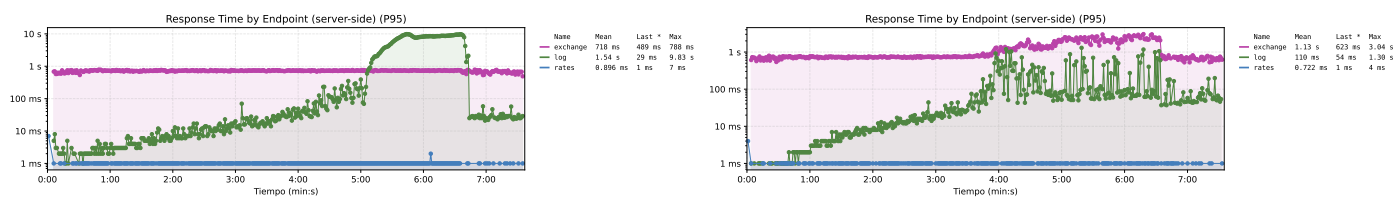


Figura 5. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Con diferencia el análisis más interesante, la propuesta 1 permitió como se esperaba mejorar la latencia en el endpoint de exchange, se puede notar que este no se ve afectado por la degradación del endpoint de logs. Como si ocurría en el caso base, donde al rededor del minuto 4, los tiempos de respuesta de exchange incrementaban, en este caso, los tiempos de respuesta se mantienen estables en 1 segundo, acotado por la dependencia del servicio externo. Por otro lado, los tiempos de respuesta de /log empeoraron, sin embargo es esperable, la mayor disponibilidad antes mencionada implica que se registraron más transacciones, por lo que el tamaño del log necesariamente aumenta. Esto explica el aumento de los tiempos de respuesta visto desde el lado del cliente antes mencionados, sin embargo, las métricas se ven ensuciadas al mezclar todos los endpoints.

4.3.3. Uso de recursos

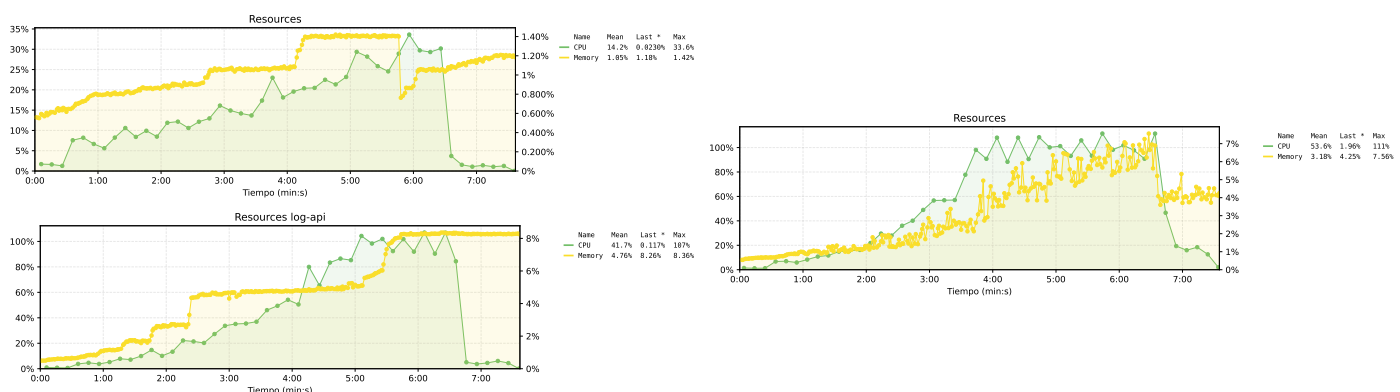


Figura 6. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Destacar que consideramos la comparación con respecto a ambos nodos, el nodo de exchange y el de logs. Se puede observar que el nodo de logs mantiene la carga con respecto a la del sistema base, incluso la aumenta. Esto es esperable con el aumento de la disponibilidad, más request completadas implican más entradas en el log. Sin embargo, el nodo de exchange apenas llegó a un 14.2 % de uso de CPU y 1.42 % de uso de memoria.

4.3.4. Volumen operado por moneda

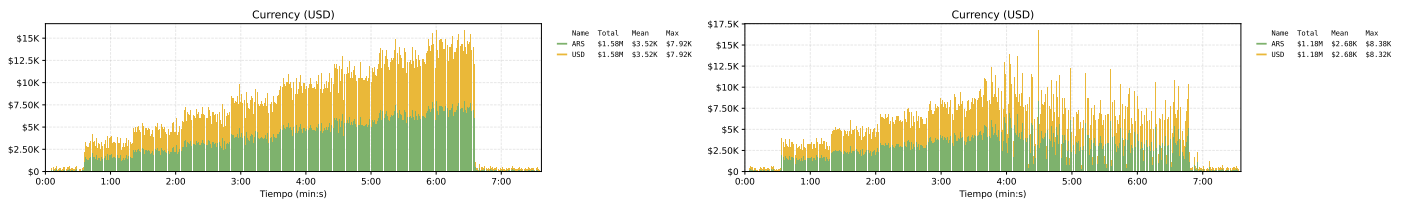


Figura 7. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 1.58M frente al 1.18M del sistema base, es decir una mejora del 33.90 %.

4.3.5. Volumen neto operado

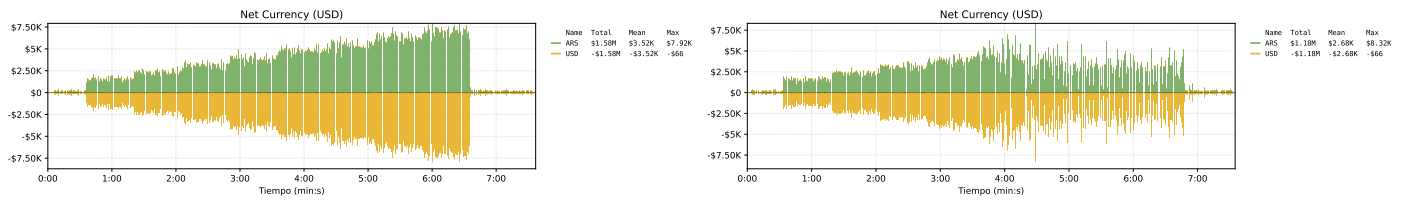


Figura 8. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Analogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base.

4.4. Pruebas de resistencia

La prueba de resistencia se realizó usando una tasa constante de 40 usuarios durante 2700s (45 minutos). Se realizó un warm-up inicial de 5 usuarios durante 60s. Además, al final de la prueba se dejó una instancia de recovery de nuevamente 5 usuarios esta vez durante 120s (2 minutos). También se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

4.4.1. Disponibilidad

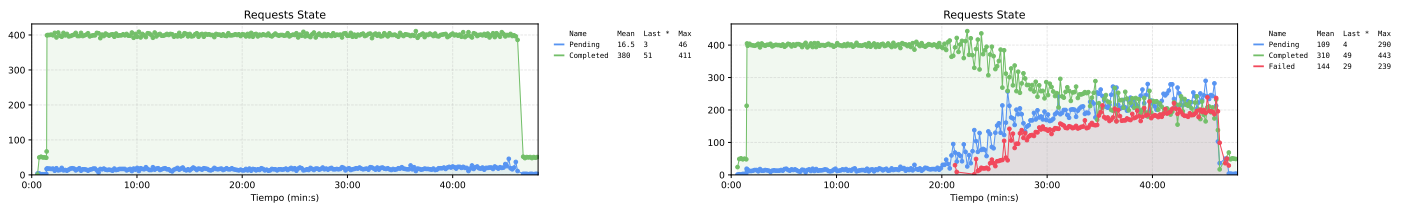


Figura 9.

TODO: completar

4.4.2. Latencia

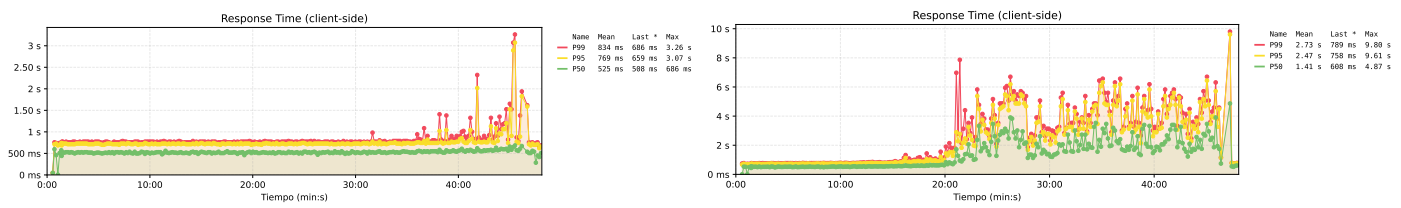


Figura 10.

TODO: completar

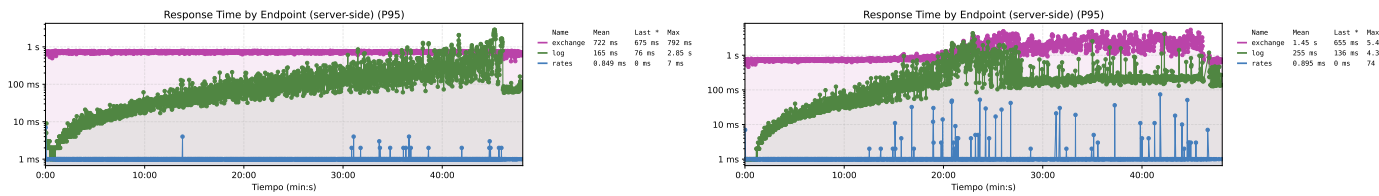


Figura 11.

TODO: completar

4.4.3. Uso de recursos

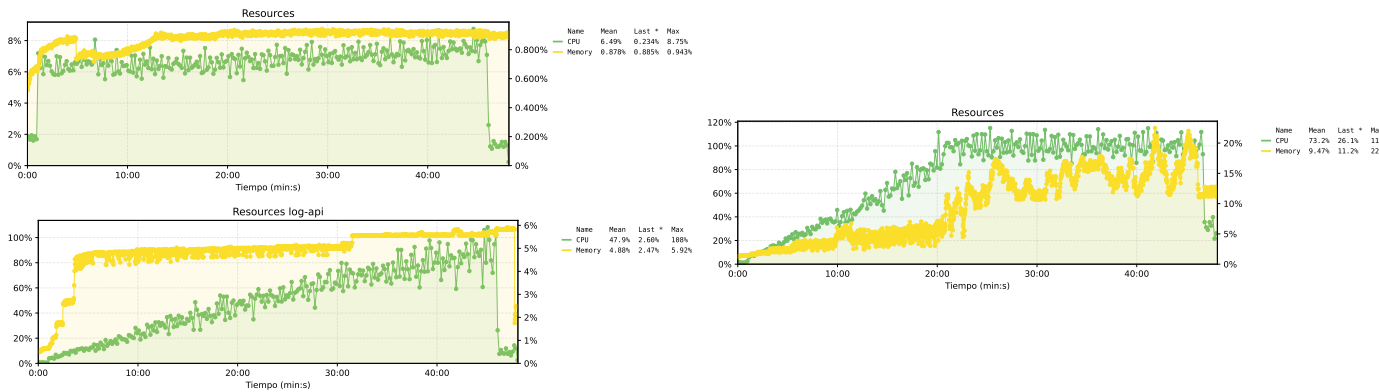


Figura 12.

TODO: completar

4.4.4. Volumen operado por moneda

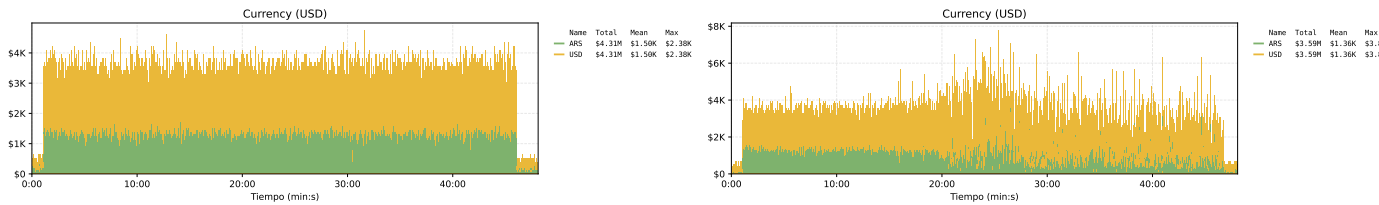


Figura 13.

TODO: completar

4.4.5. Volumen neto operado

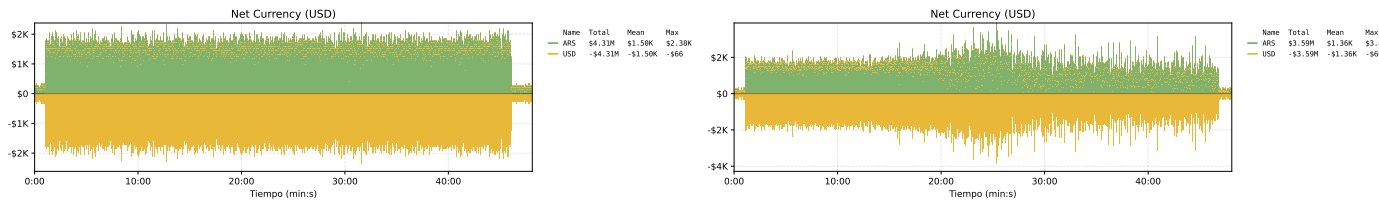


Figura 14.

TODO: completar

4.5. Resultados y conclusiones

La propuesta 1 demostro ser capaz de aislar la degradacion producida por la consulta de logs de las opeeraciones de negocio del sistema, permitiendo ofrecer una mejor disponibilidad a los usuarios comunes. Por otro lado la consulta de logs sigue degradandose con el tiempo.

Sinembargo, este desacoplamiento propuesto permitiría replicar el nodo de consulta para dar soporte de lectura de los a mas usuarios, es decir, permitiendo escalabilidad horizontal.

5. PROPUESTA 2

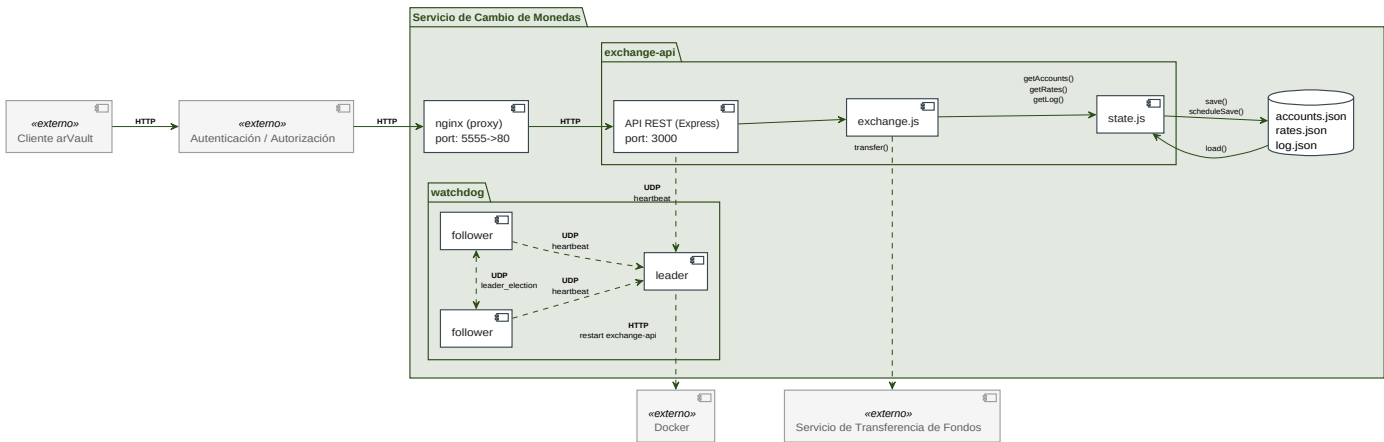


Figura 15. Diagrama Components & Connectors de la propuesta 2.

5.1. Impacto en los QA

5.1.1. Performance

5.2. Pruebas de carga

Se pueden encontrar los resultados completos en el Apéndice.

5.3. Pruebas de estrés

Se pueden encontrar los resultados completos en el Apéndice.

5.3.1. Disponibilidad

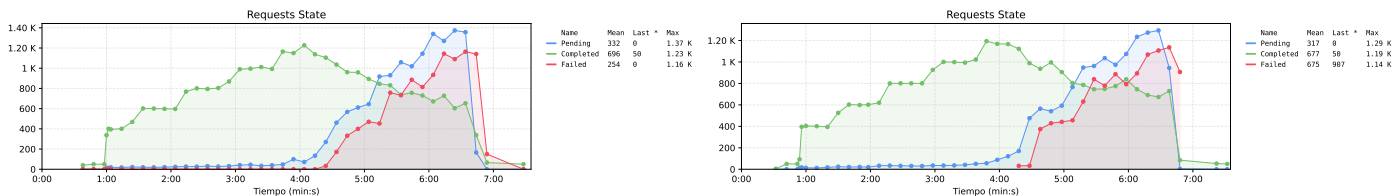


Figura 16.

TODO: completar

5.3.2. Latencia

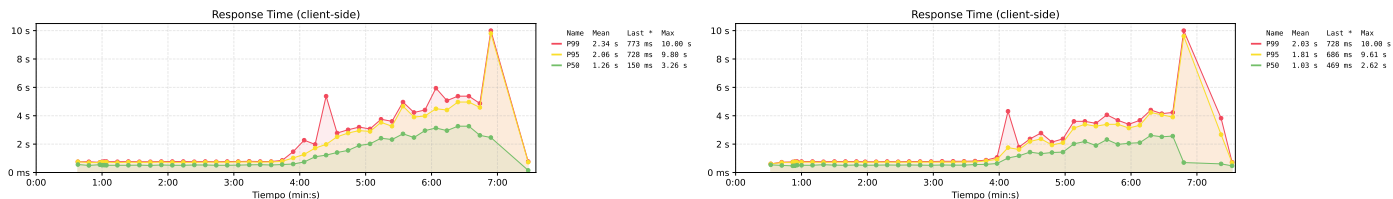


Figura 17.

TODO: completar

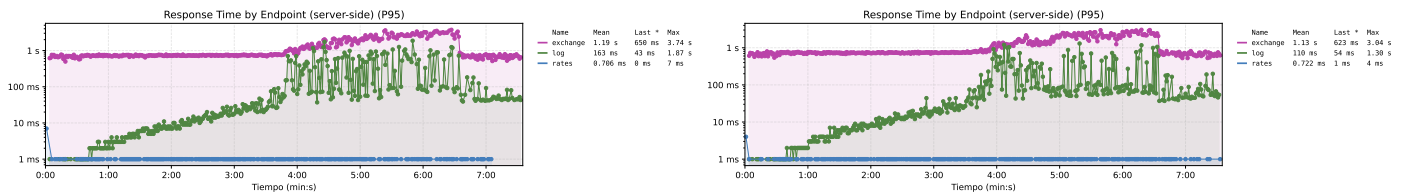


Figura 18.

TODO: completar

5.3.3. Uso de recursos

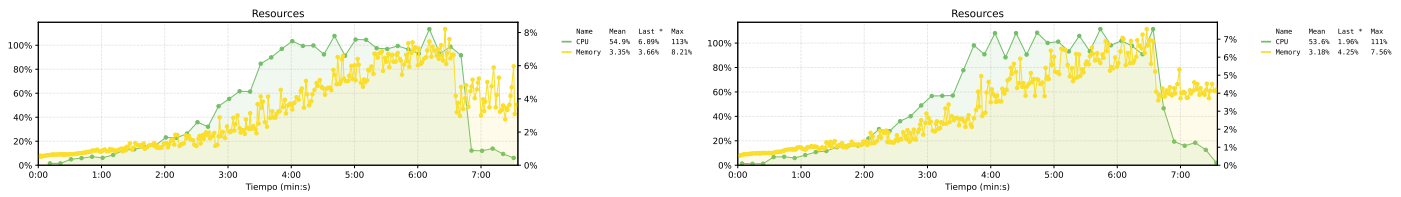


Figura 19.

TODO: completar

5.3.4. Volumen operado por moneda

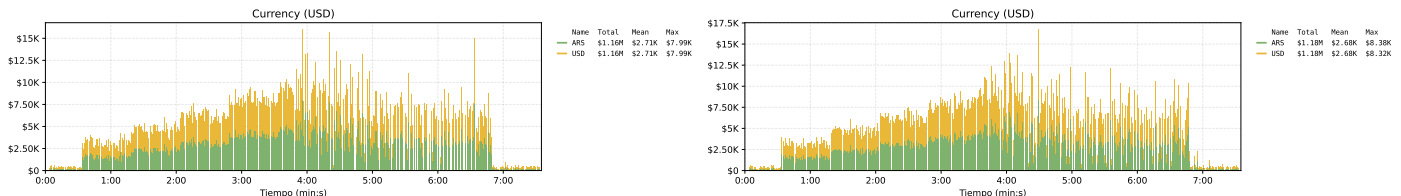


Figura 20.

TODO: completar

5.3.5. Volumen neto operado

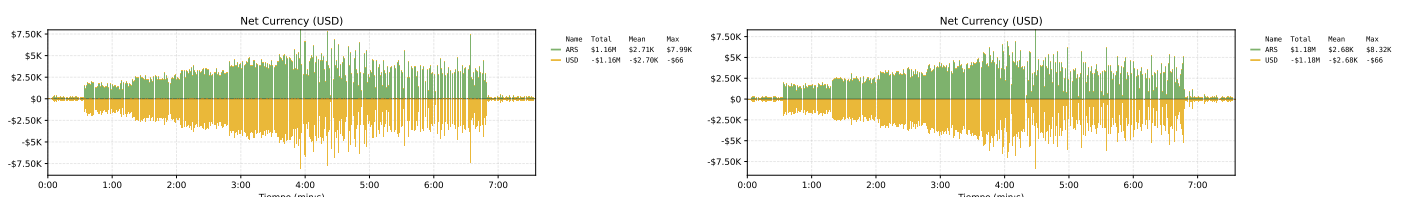


Figura 21.

TODO: completar

5.4. Pruebas de resistencia

Se pueden encontrar los resultados completos en el Apéndice.

5.4.1. Disponibilidad

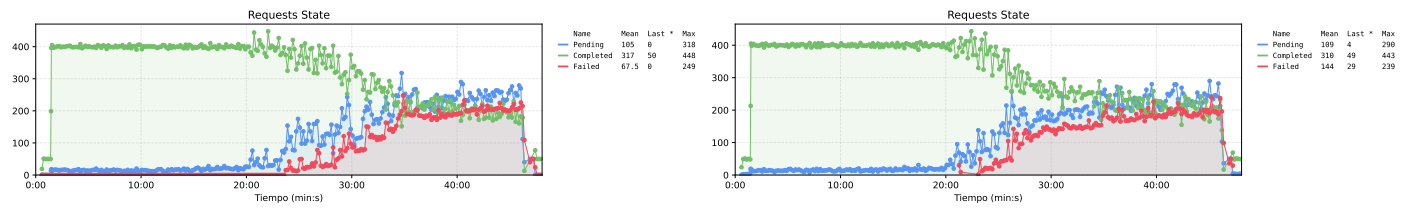


Figura 22.

TODO: completar

5.4.2. Latencia

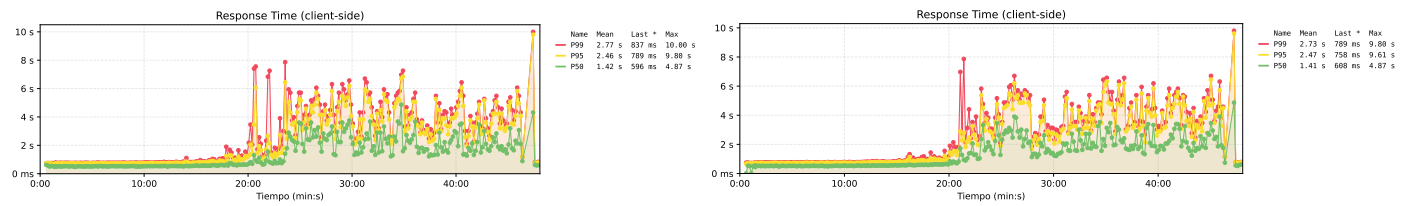


Figura 23.

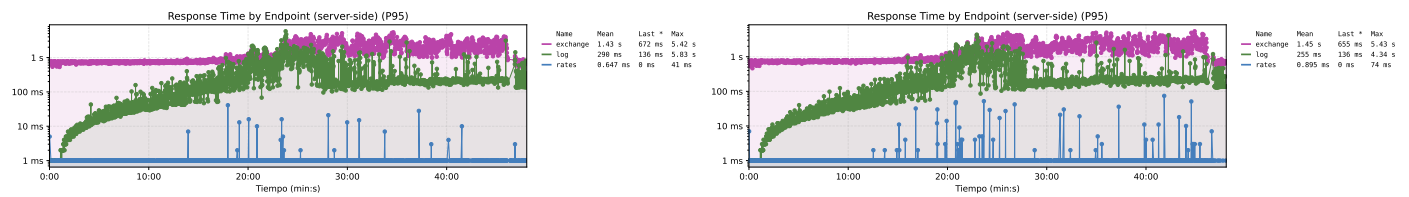
TODO: completar

Figura 24.

TODO: completar

5.4.3. Uso de recursos

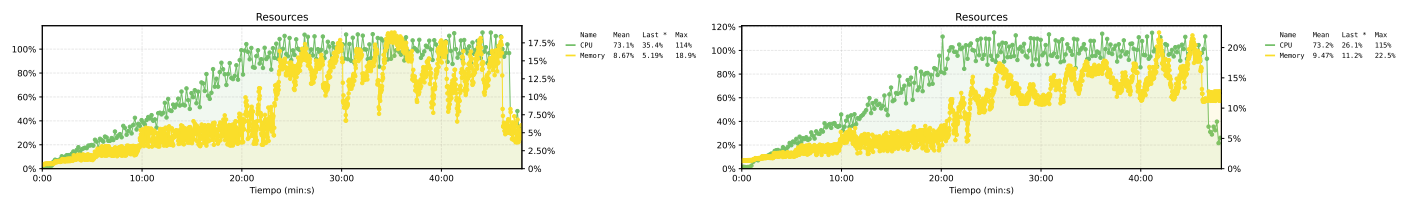


Figura 25.

TODO: completar

5.4.4. Volumen operado por moneda

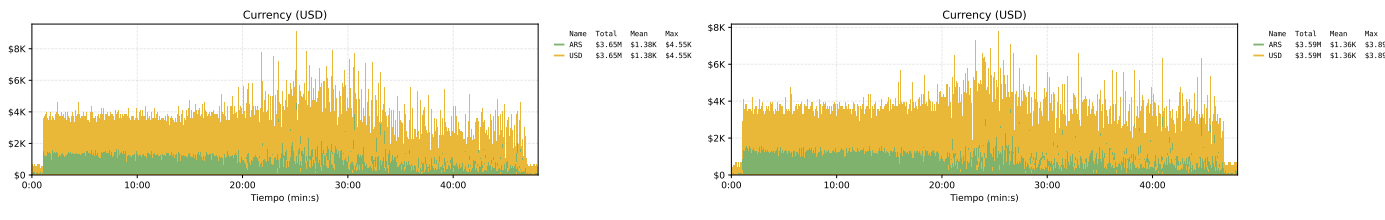


Figura 26.

TODO: completar

5.4.5. Volumen neto operado

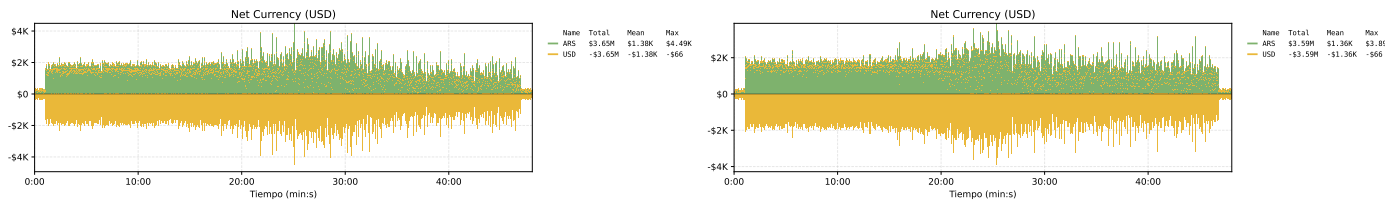


Figura 27.

TODO: completar

5.5. Pruebas de caos

TODO: describir el escenario

Se pueden encontrar los resultados completos en el [Apendice](#).

5.5.1. Disponibilidad

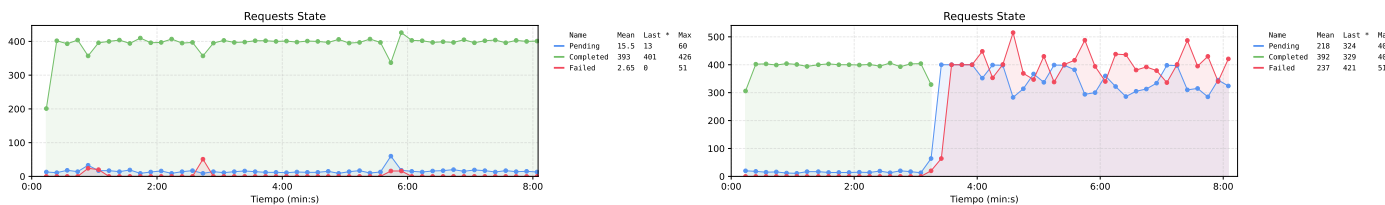


Figura 28.

TODO: completar

5.5.2. Latencia

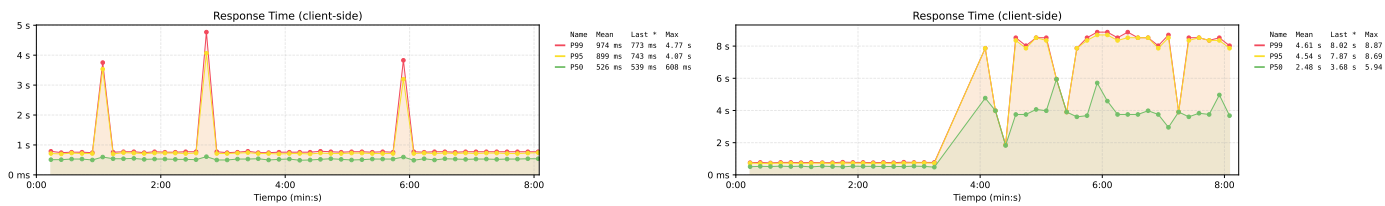


Figura 29.

TODO: completar

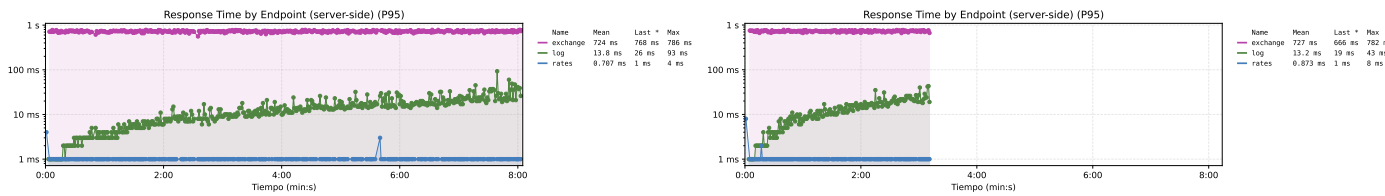


Figura 30.

TODO: completar

5.5.3. Uso de recursos

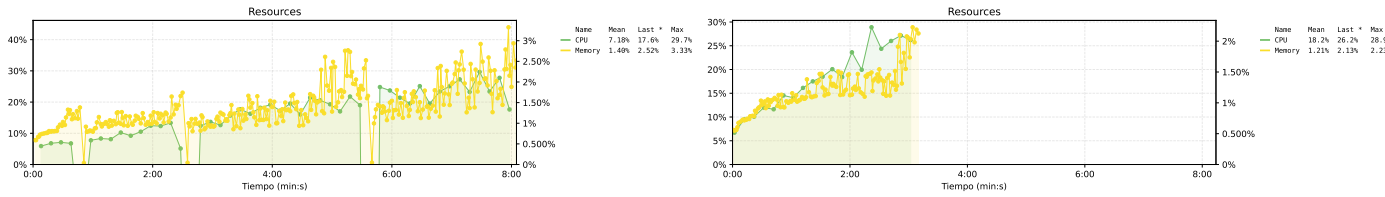


Figura 31.

TODO: completar

5.5.4. Volumen operado por moneda

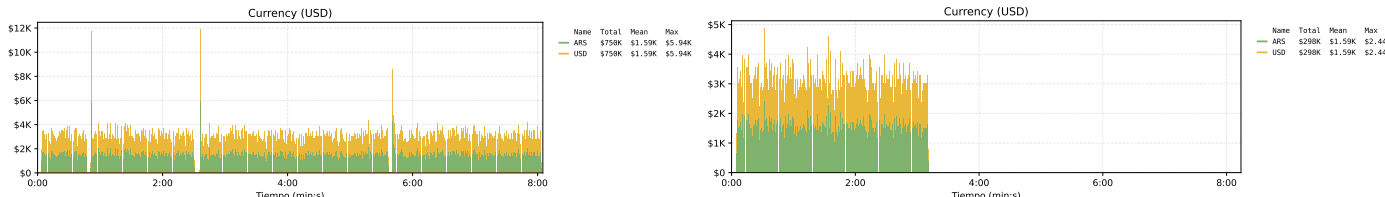


Figura 32.

TODO: completar

5.5.5. Volumen neto operado

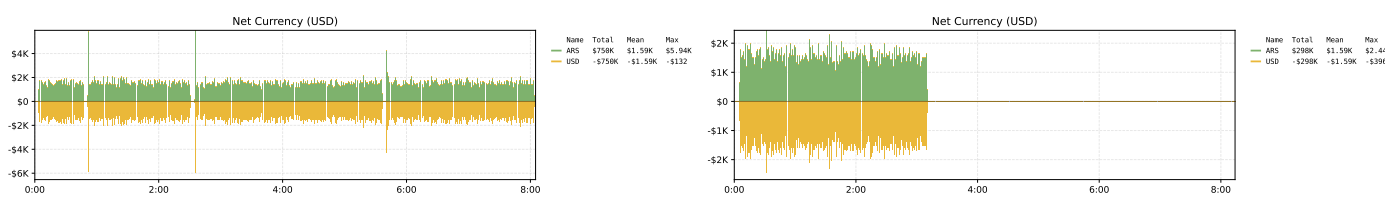


Figura 33.

TODO: completar

5.6. Resultados y conclusiones

6. USO DE HERRAMIENTAS DE IA

...

A. APENDICE

A.1. Resultados completos del escenario de carga sobre el sistema base

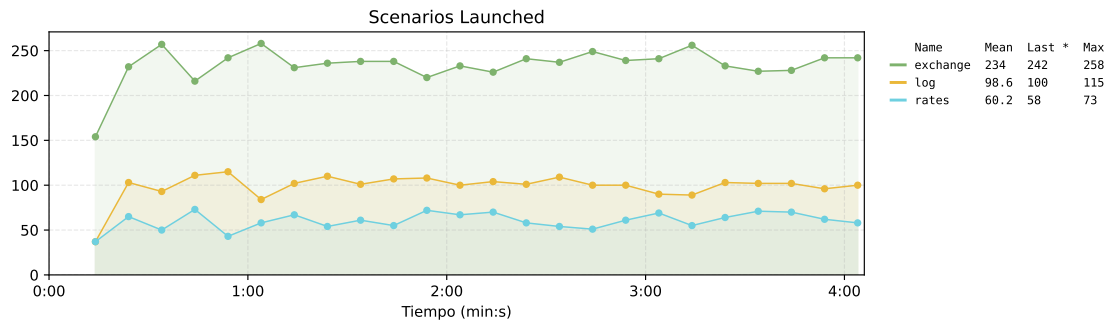


Figura 34. Escenarios del sistema base.

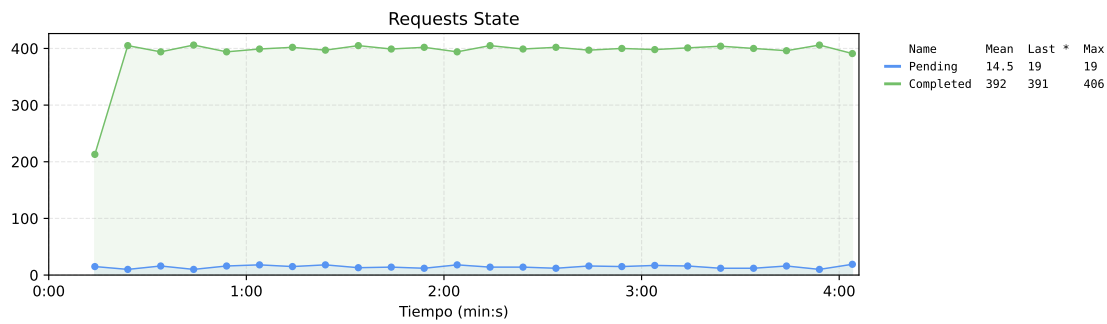


Figura 35. Estado de las requests del sistema base.

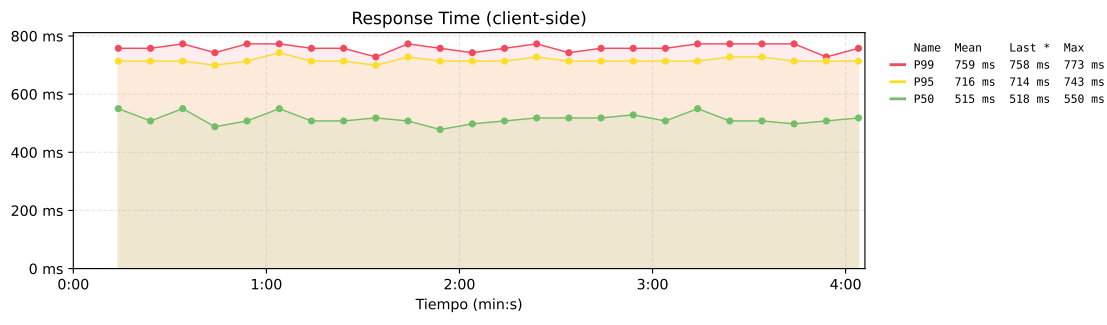


Figura 36. Tiempo de respuesta del sistema base.

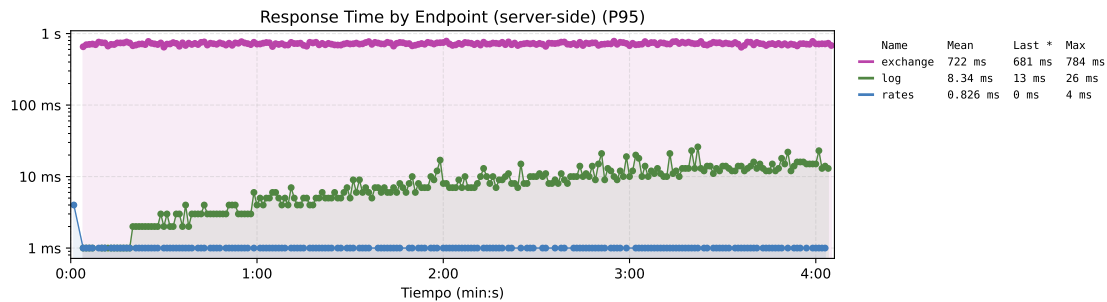


Figura 37. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

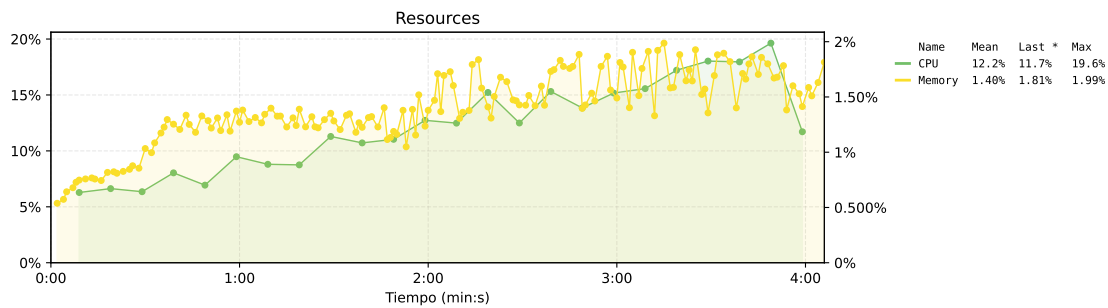


Figura 38. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

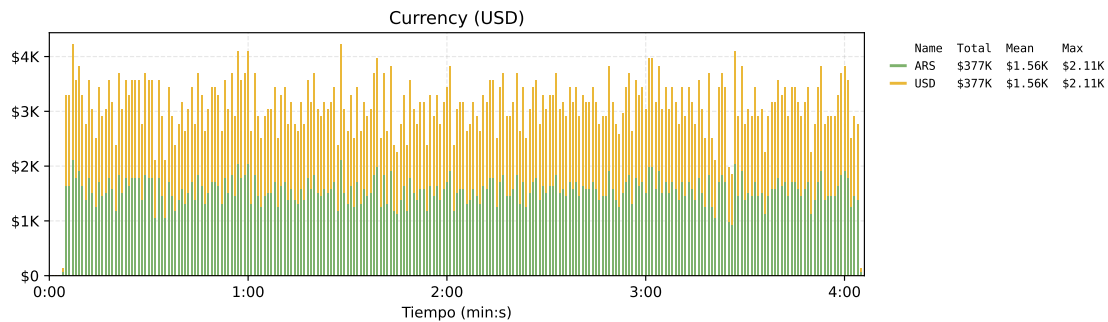


Figura 39. Volumen operado en cada moneda en el sistema base.

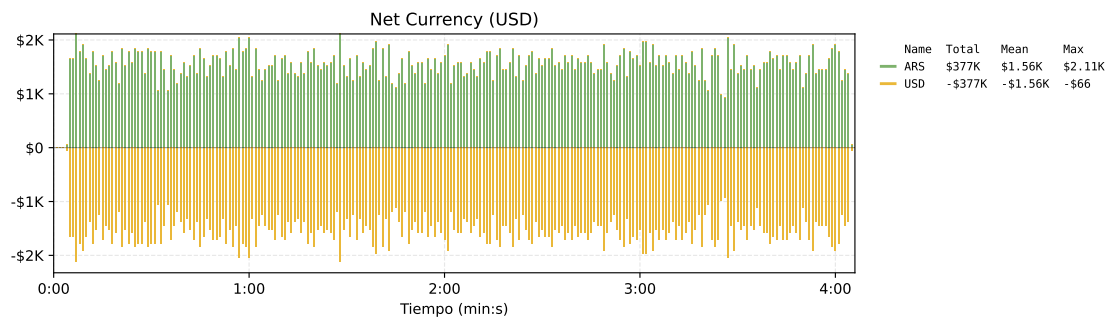


Figura 40. Neto operado en cada moneda en el sistema base.

A.2. Resultados completos del escenario de estrés sobre el sistema base

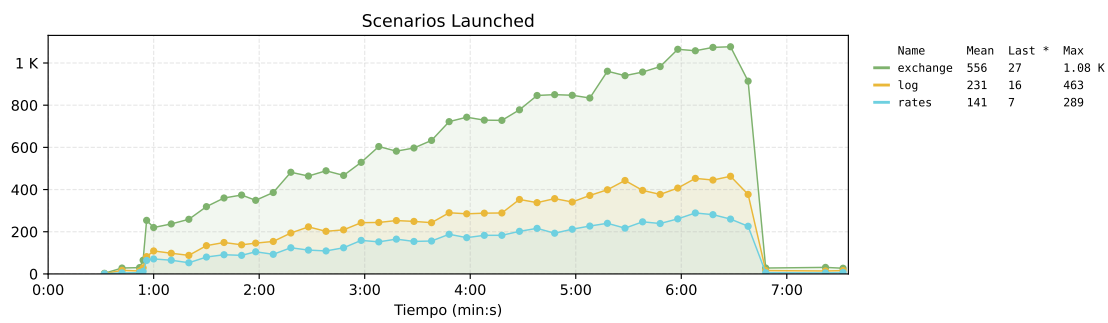


Figura 41. Escenarios del sistema base.

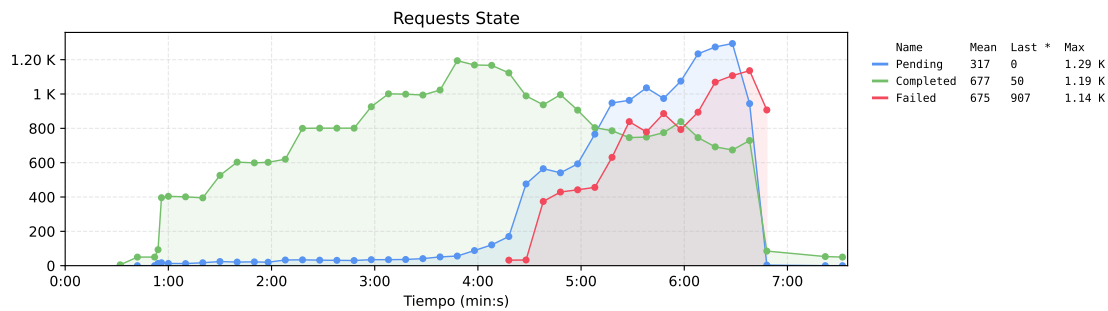


Figura 42. Estado de las requests del sistema base.

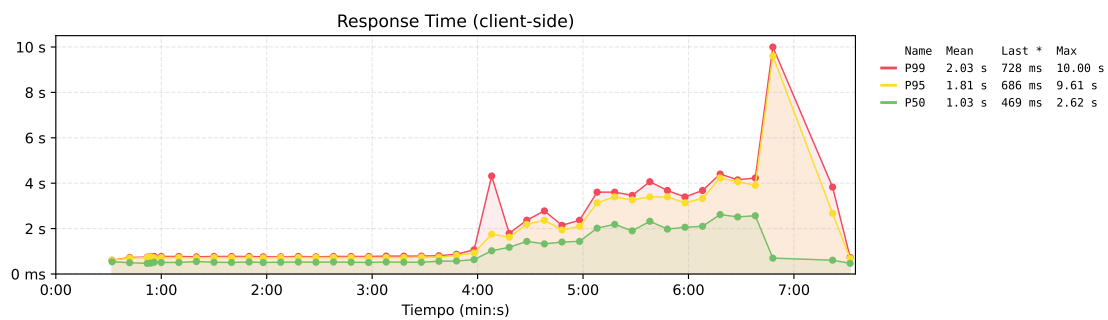


Figura 43. Tiempo de respuesta del sistema base.

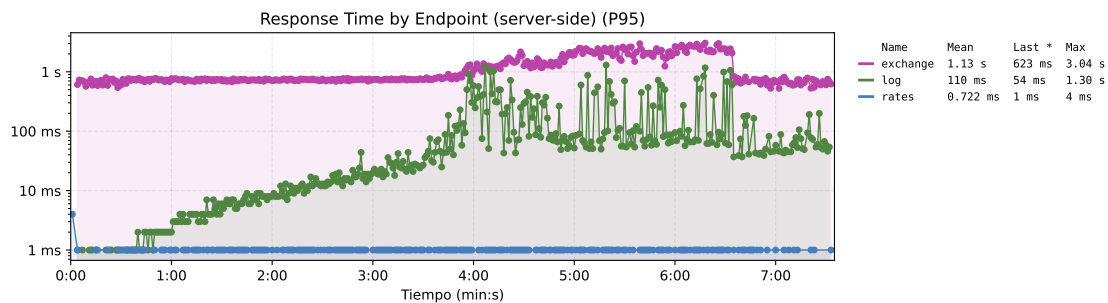


Figura 44. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

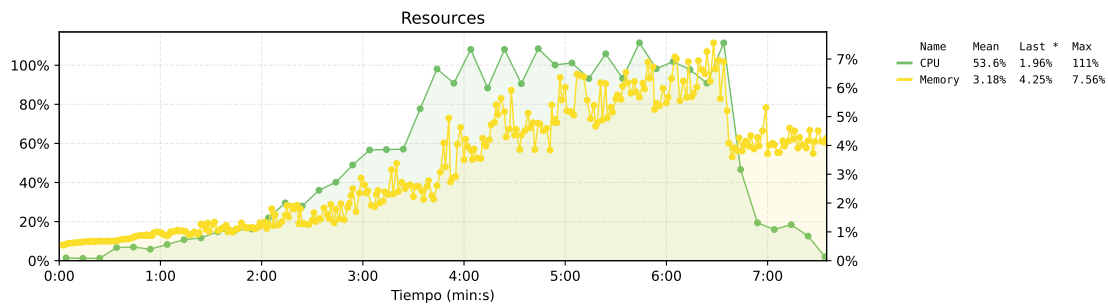


Figura 45. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

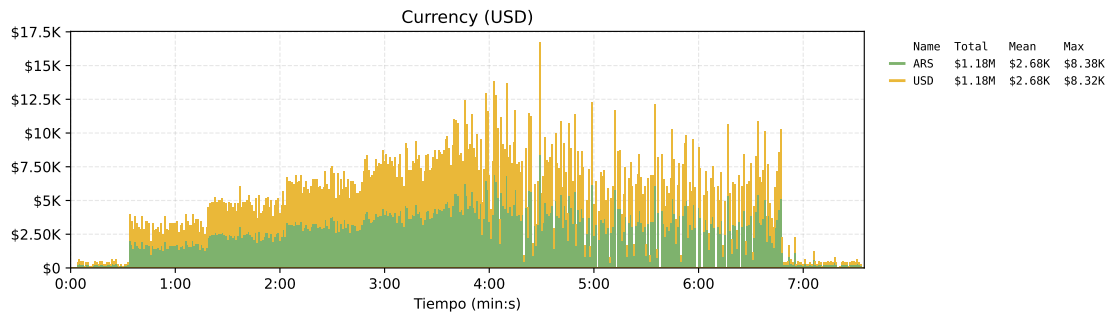


Figura 46. Volumen operado en cada moneda en el sistema base.

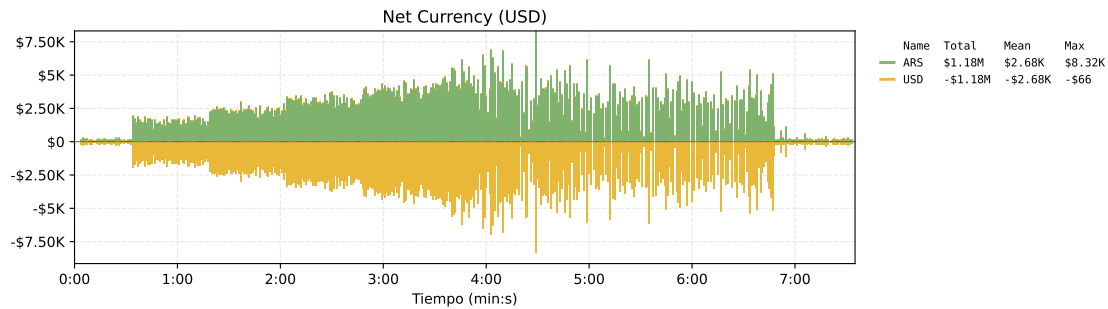


Figura 47. Neto operado en cada moneda en el sistema base.

A.3. Resultados completos del escenario de resistencia sobre el sistema base

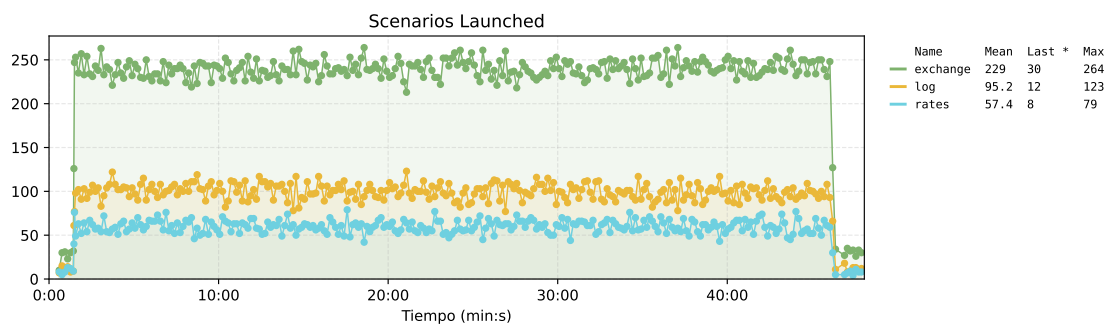


Figura 48. Escenarios del sistema base.

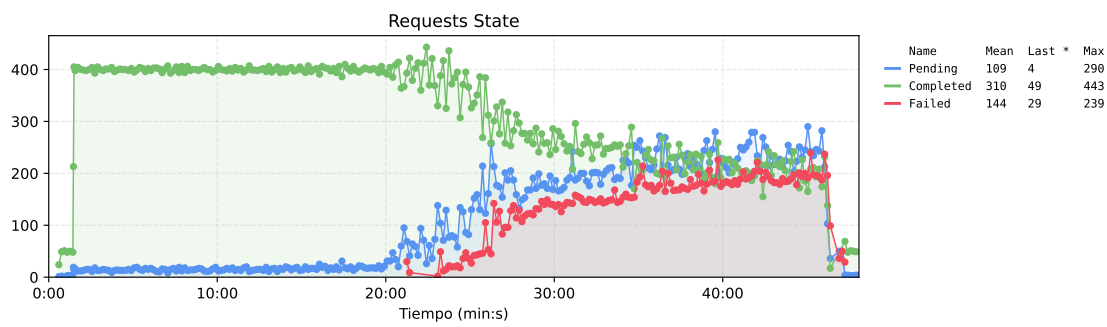


Figura 49. Estado de las requests del sistema base.

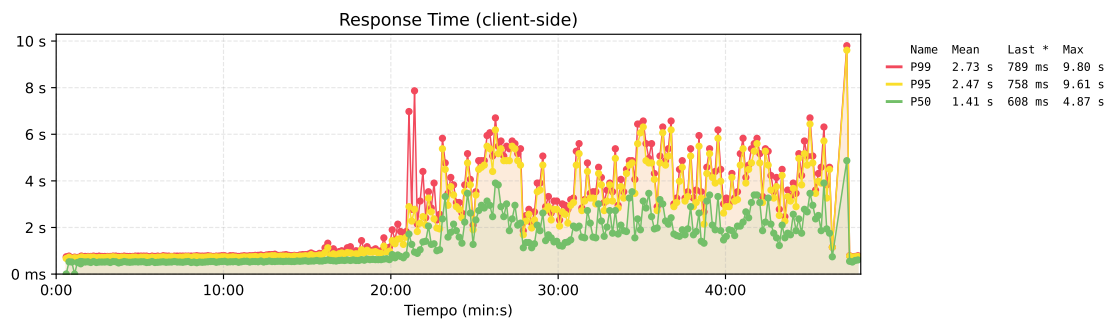


Figura 50. Tiempo de respuesta del sistema base.

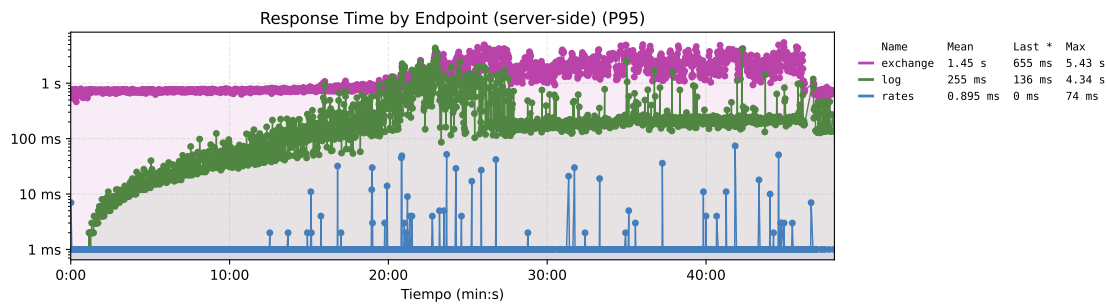


Figura 51. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

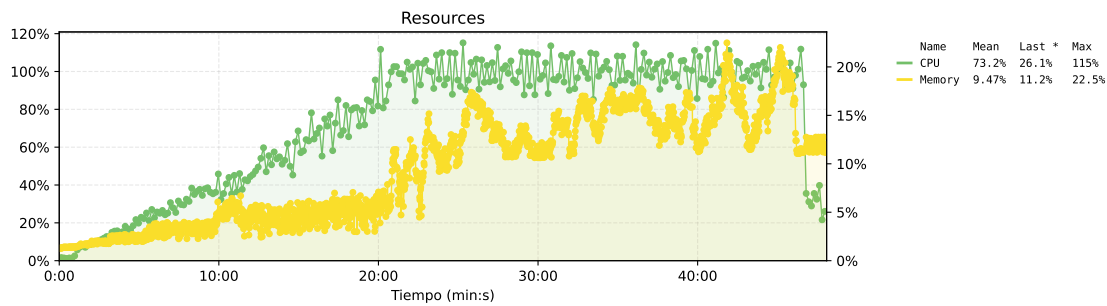


Figura 52. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

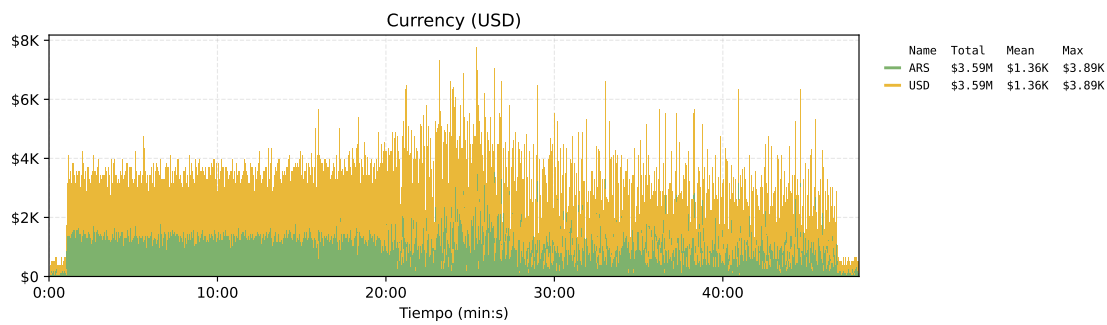


Figura 53. Volumen operado en cada moneda en el sistema base.

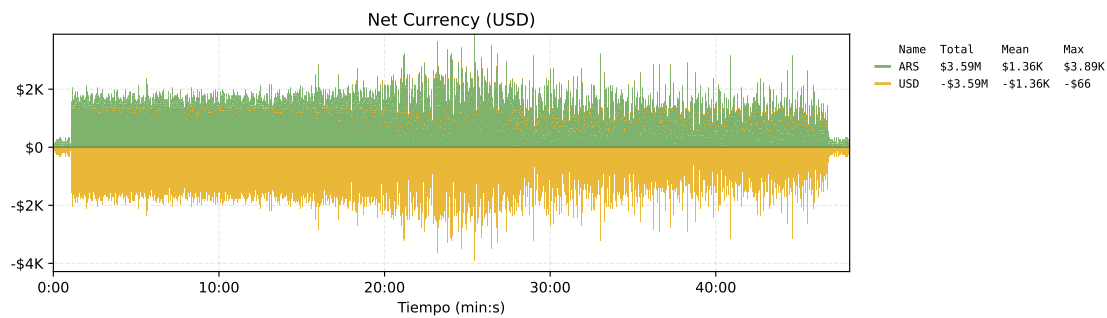


Figura 54. Neto operado en cada moneda en el sistema base.

A.4. Resultados completos del escenario de caos sobre el sistema base

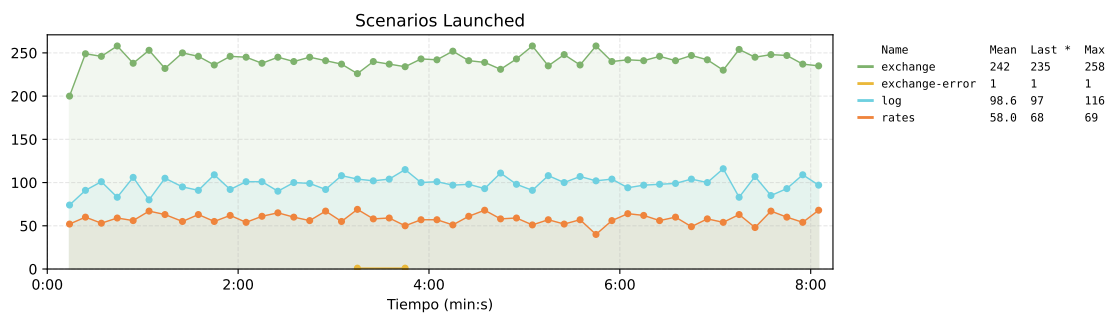


Figura 55. Escenarios del sistema base.

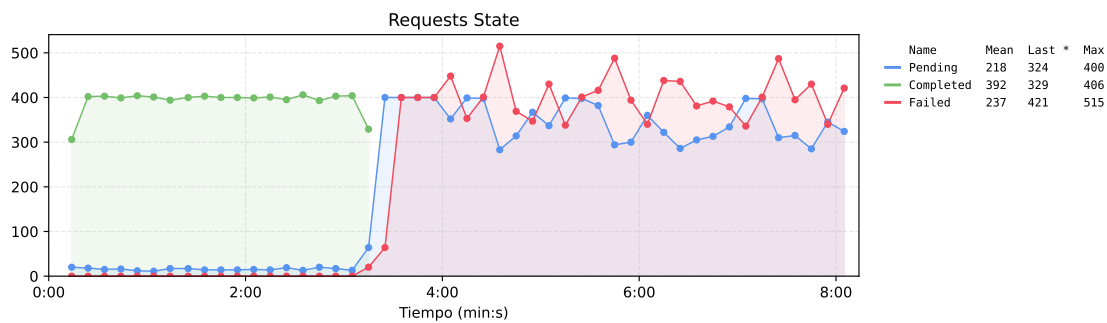


Figura 56. Estado de las requests del sistema base.

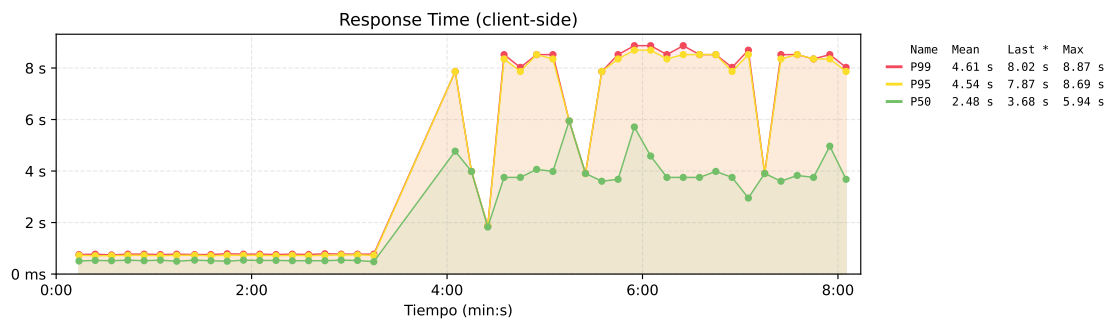


Figura 57. Tiempo de respuesta del sistema base.

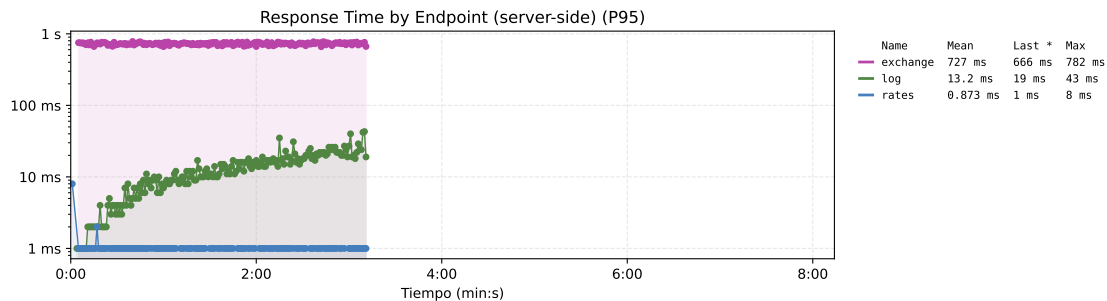


Figura 58. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

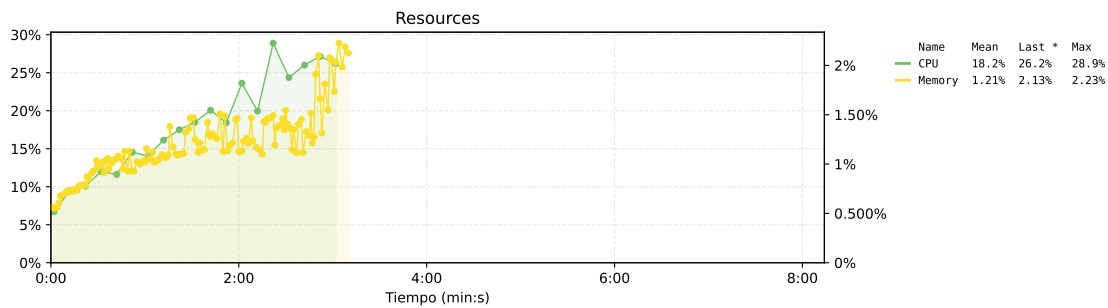


Figura 59. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

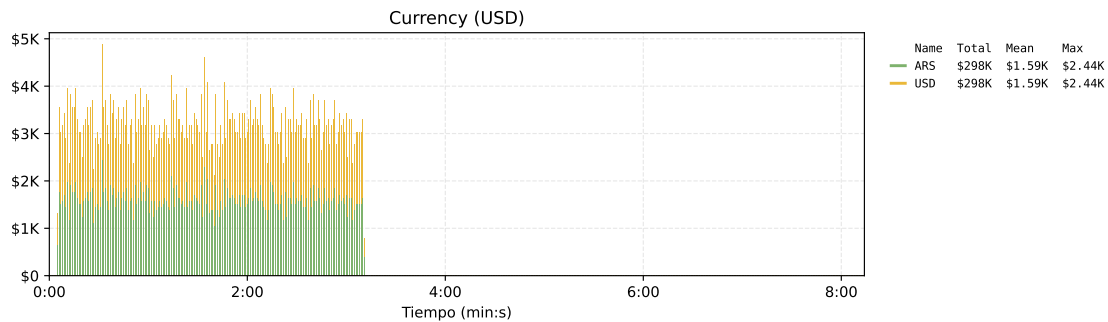


Figura 60. Volumen operado en cada moneda en el sistema base.

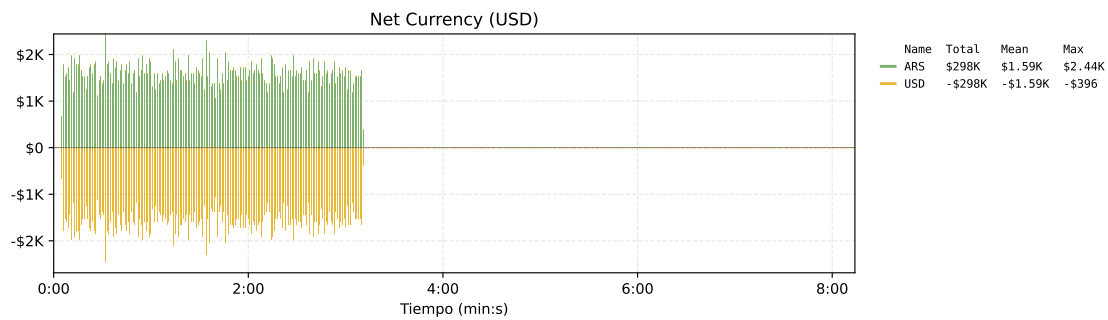


Figura 61. Neto operado en cada moneda en el sistema base.

A.5. Resultados completos del escenario de carga sobre la propuesta 1

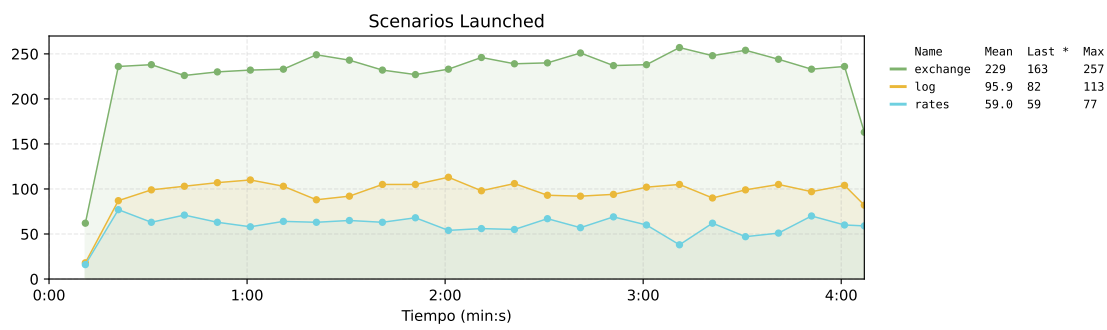


Figura 62. Escenarios de la propuesta 1.

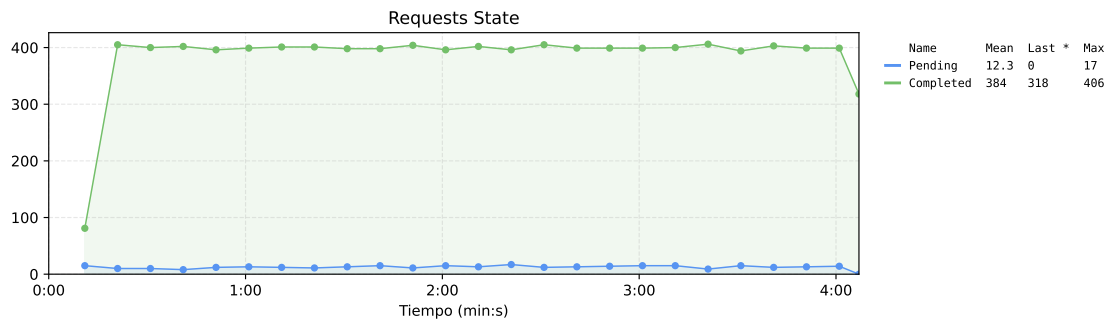


Figura 63. Estado de las requests de la propuesta 1.

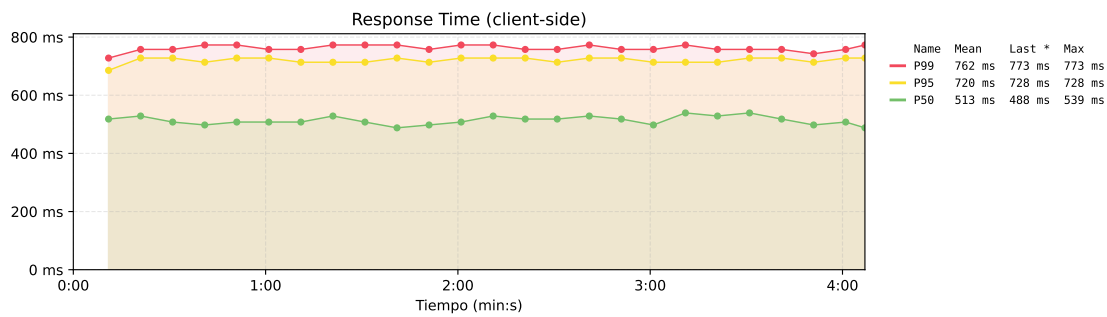


Figura 64. Tiempo de respuesta de la propuesta 1.

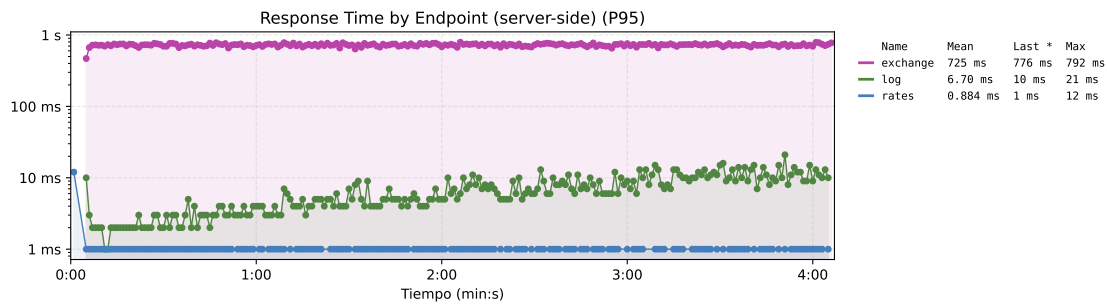


Figura 65. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logaritmica.

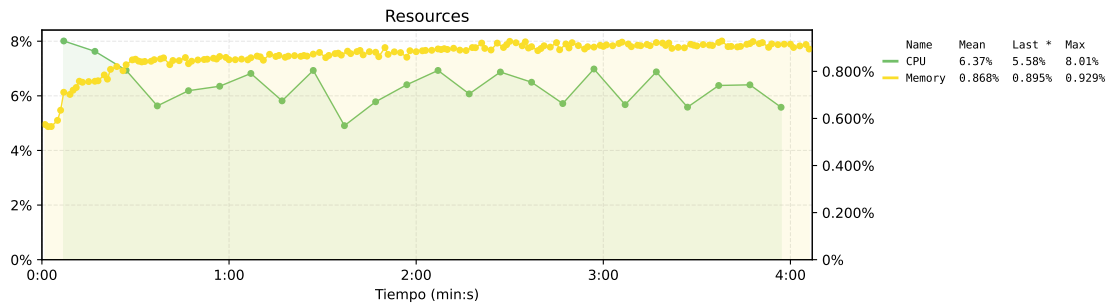


Figura 66. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

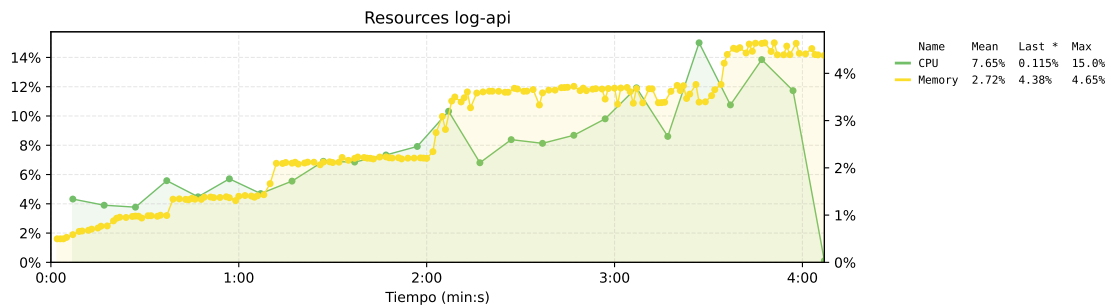


Figura 67. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

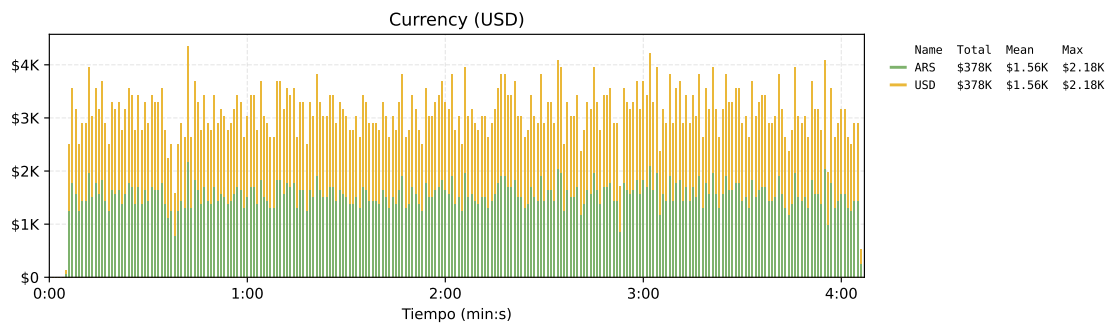


Figura 68. Volumen operado en cada moneda en la propuesta 1.

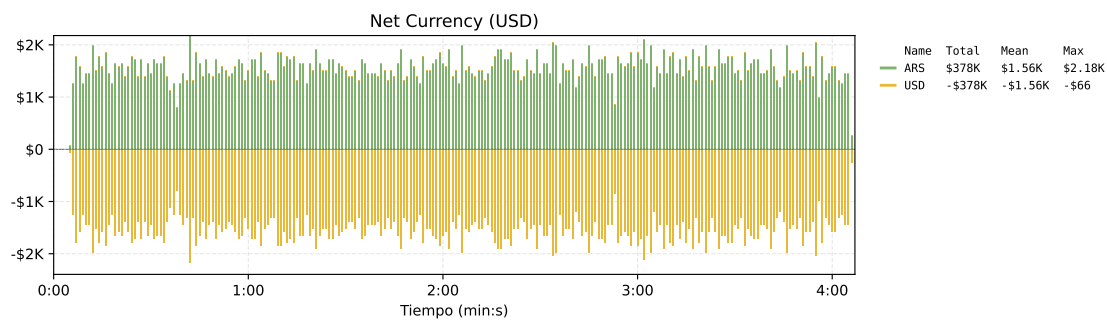


Figura 69. Neto operado en cada moneda en la propuesta 1.

A.6. Resultados completos del escenario de estrés sobre la propuesta 1

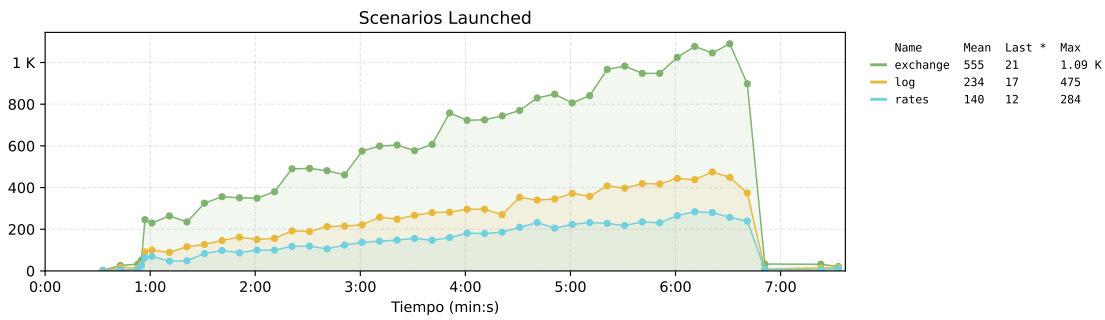


Figura 70. Escenarios de la propuesta 1.

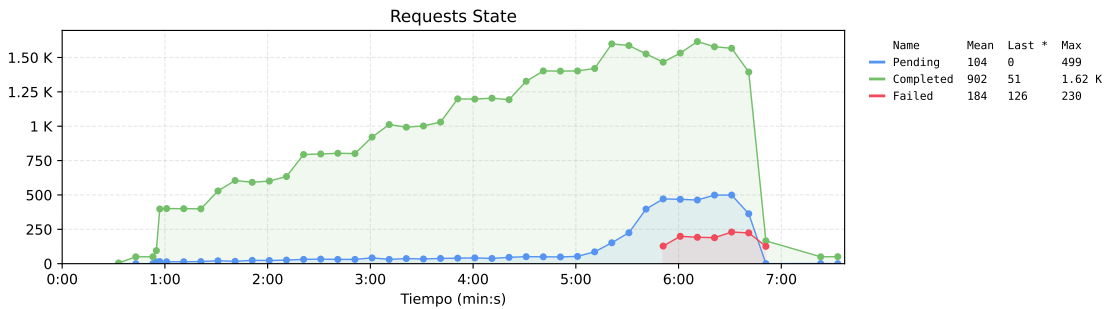


Figura 71. Estado de las requests de la propuesta 1.

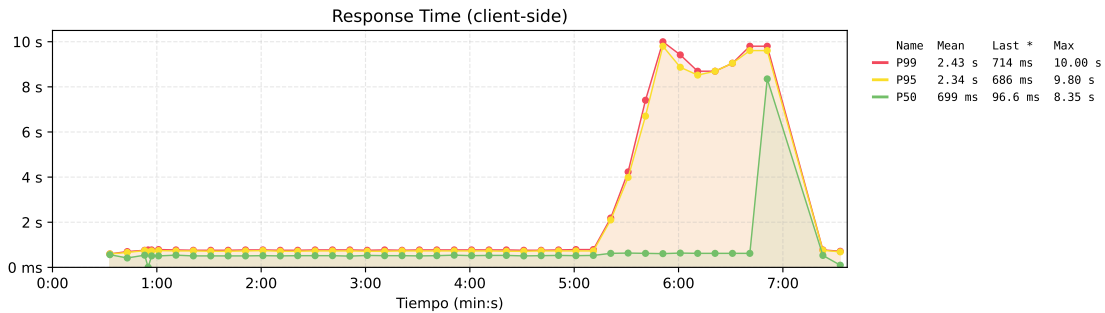


Figura 72. Tiempo de respuesta de la propuesta 1.

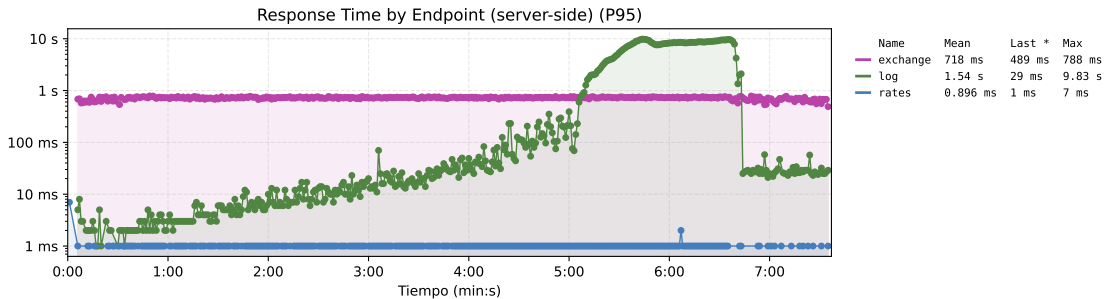


Figura 73. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

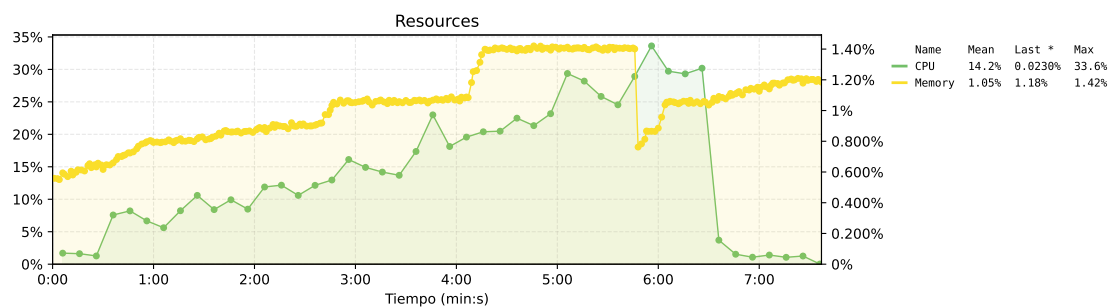


Figura 74. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

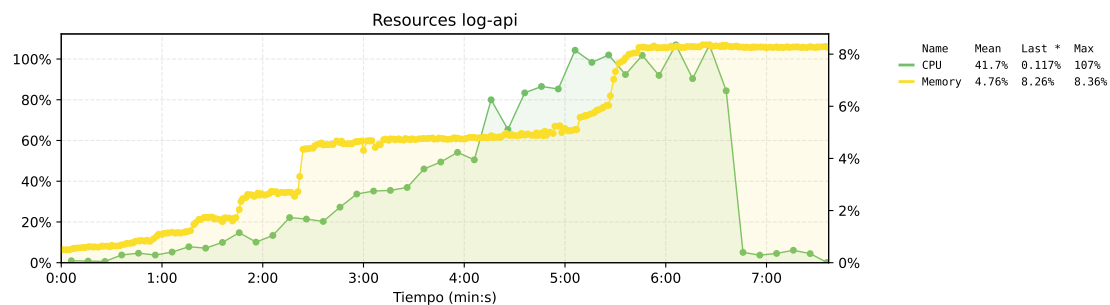


Figura 75. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

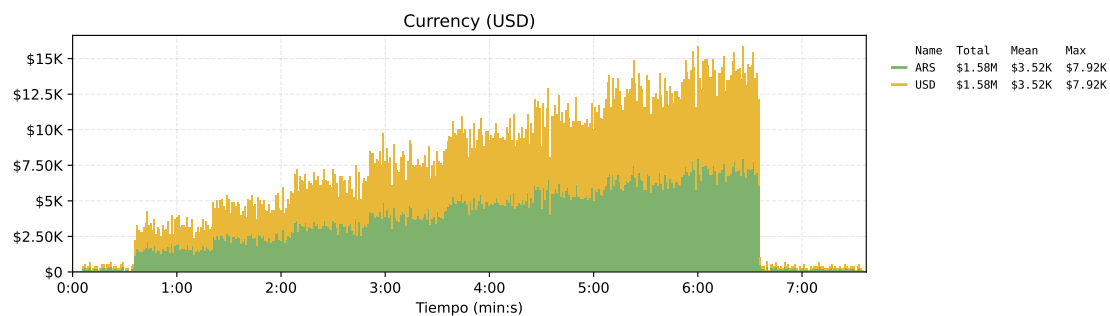


Figura 76. Volumen operado en cada moneda en la propuesta 1.

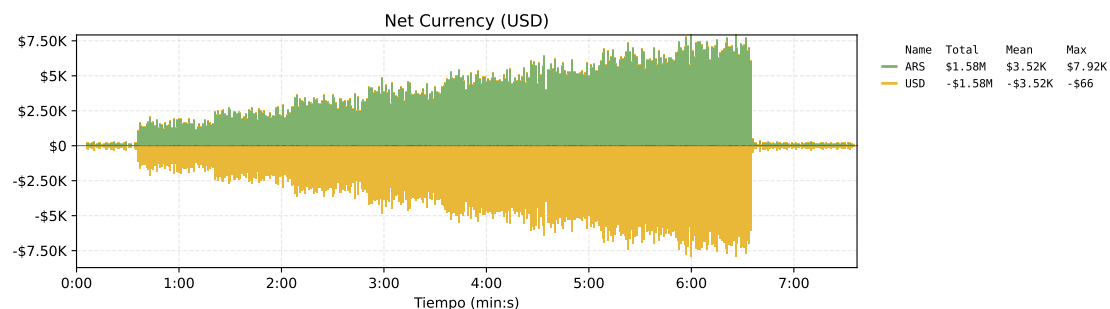


Figura 77. Neto operado en cada moneda en la propuesta 1.

A.7. Resultados completos del escenario de resistencia sobre la propuesta 1

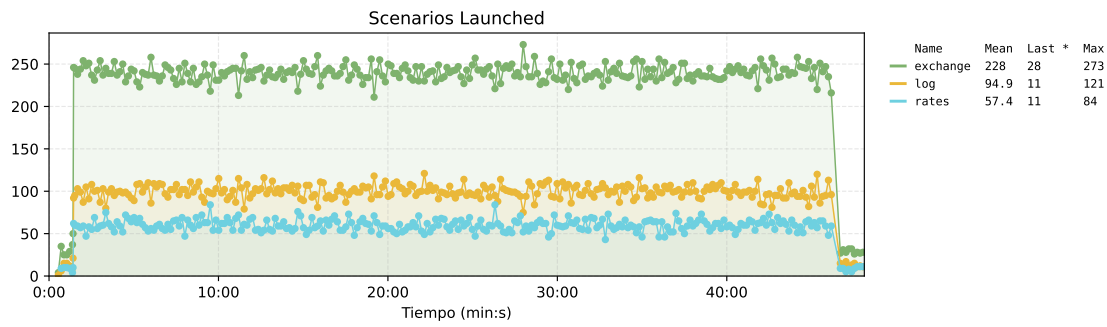


Figura 78. Escenarios de la propuesta 1.

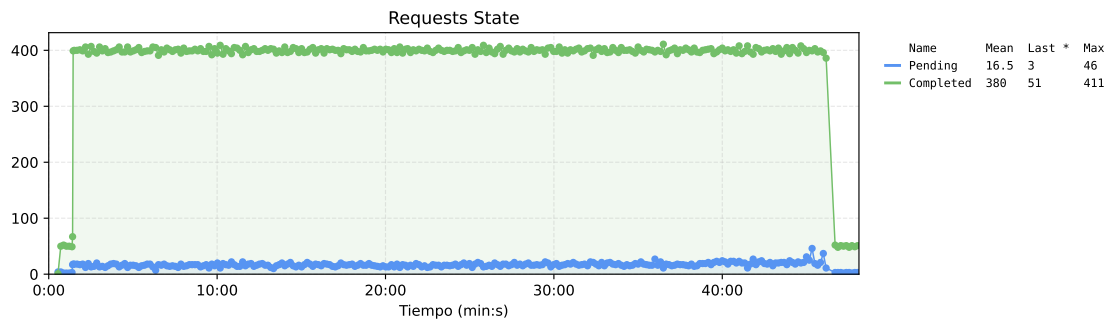


Figura 79. Estado de las requests de la propuesta 1.

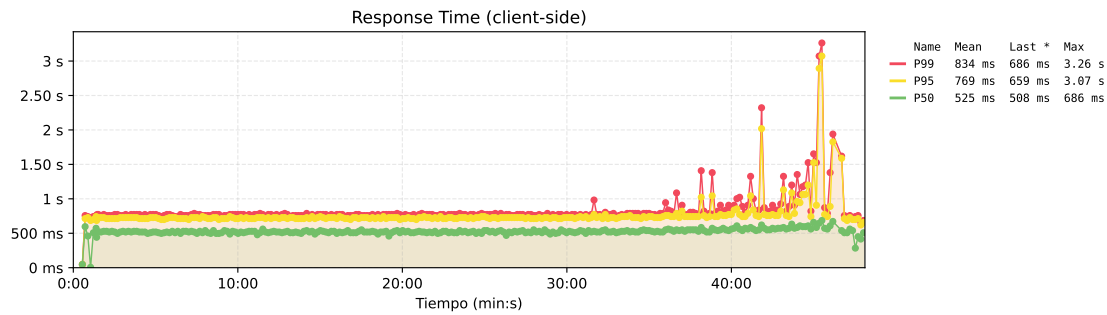


Figura 80. Tiempo de respuesta de la propuesta 1.

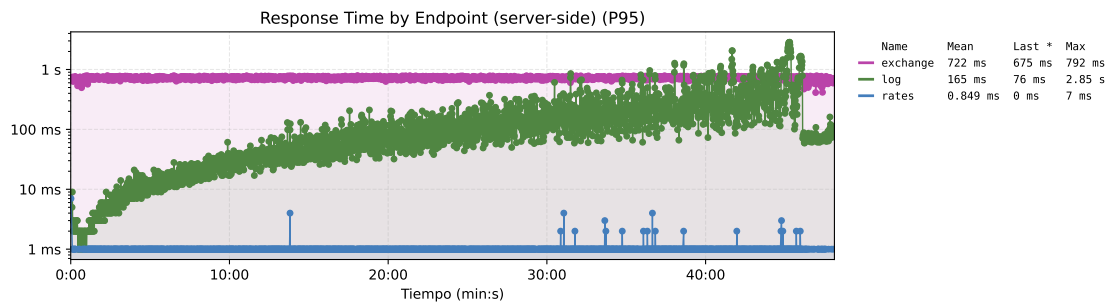


Figura 81. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

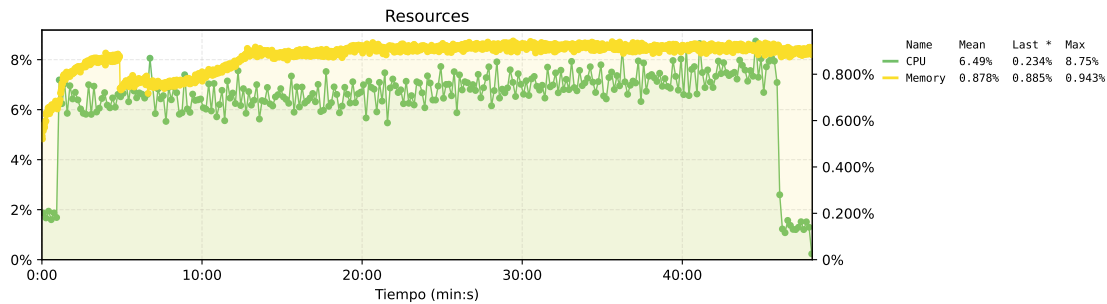


Figura 82. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

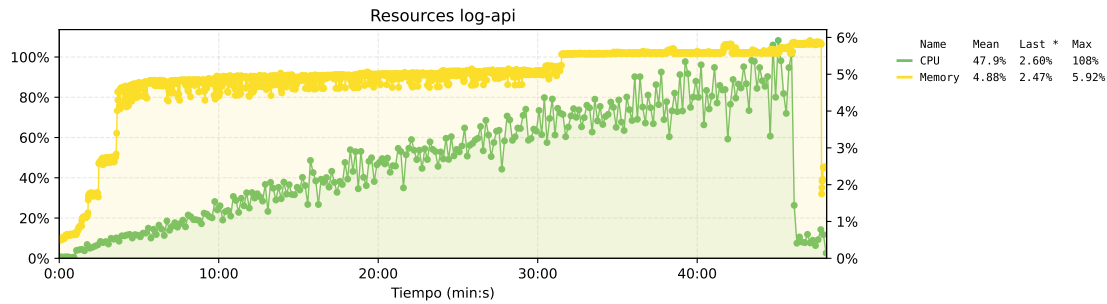


Figura 83. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

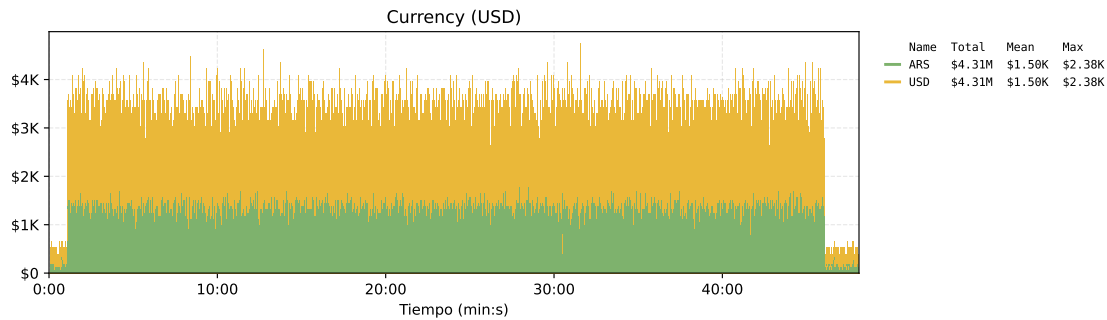


Figura 84. Volumen operado en cada moneda en la propuesta 1.

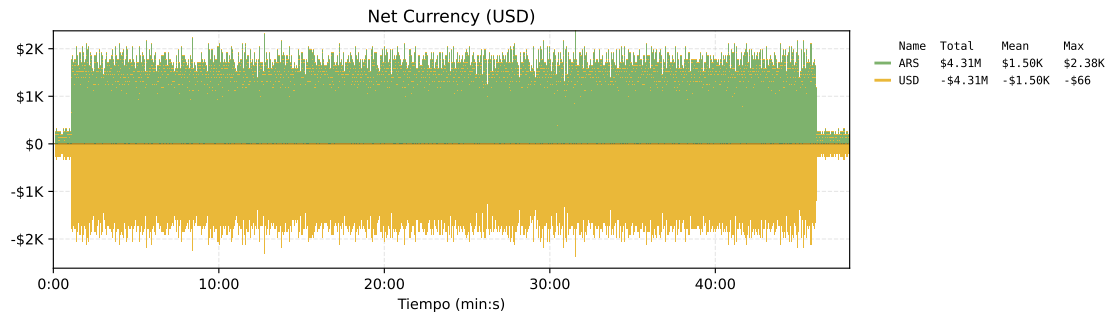


Figura 85. Neto operado en cada moneda en la propuesta 1.

A.8. Resultados completos del escenario de carga sobre la propuesta 2

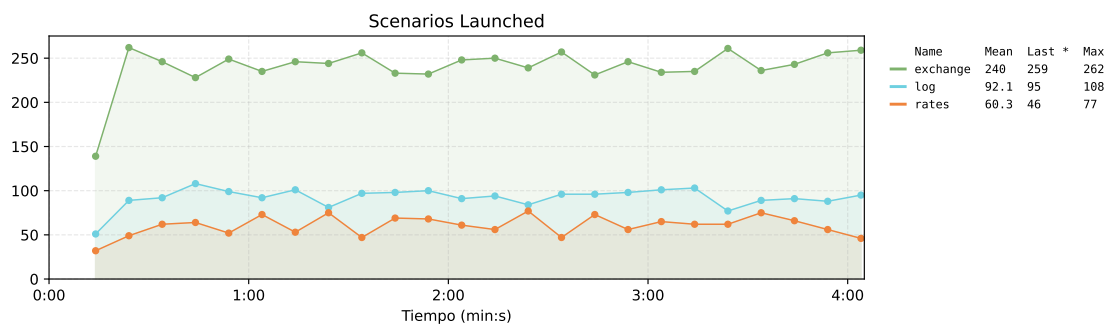


Figura 86. Escenarios de la propuesta 2.

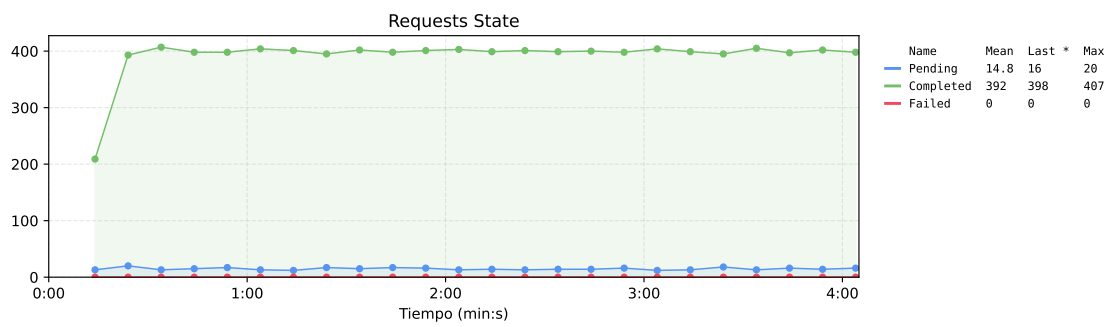


Figura 87. Estado de las requests de la propuesta 2.

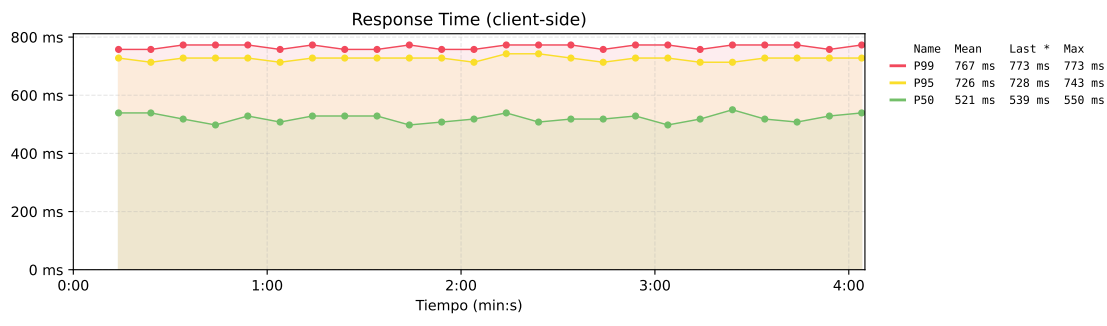


Figura 88. Tiempo de respuesta de la propuesta 2.

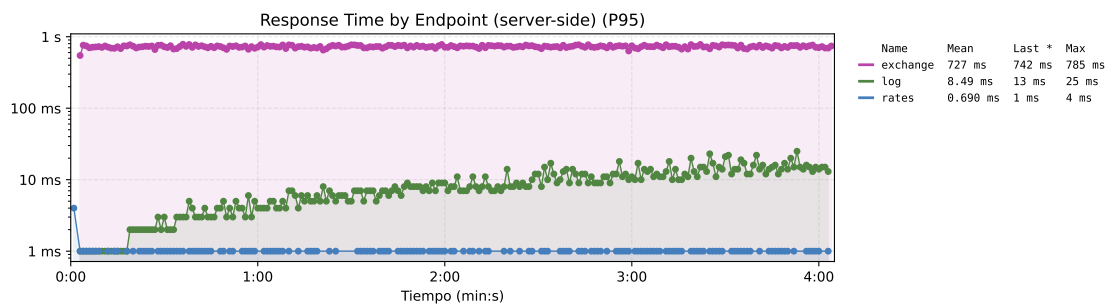


Figura 89. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logaritmica.

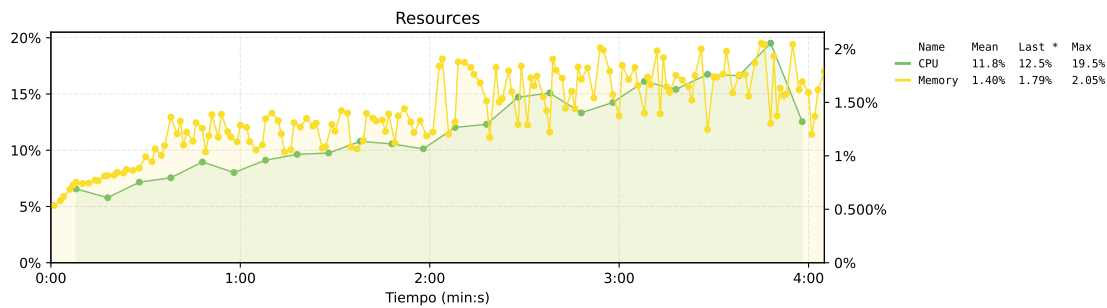


Figura 90. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

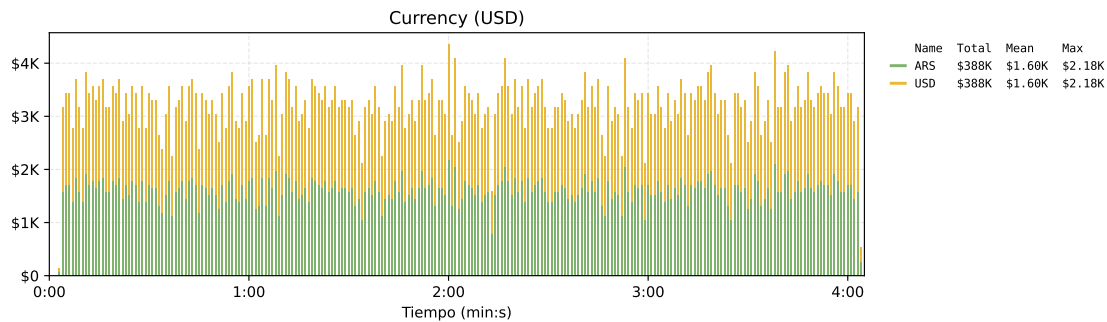


Figura 91. Volumen operado en cada moneda en la propuesta 2.

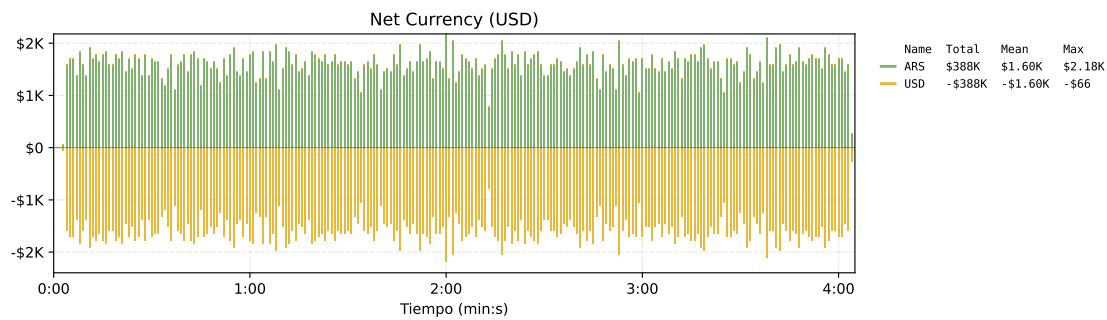


Figura 92. Neto operado en cada moneda en la propuesta 2.

A.9. Resultados completos del escenario de estrés sobre la propuesta 2

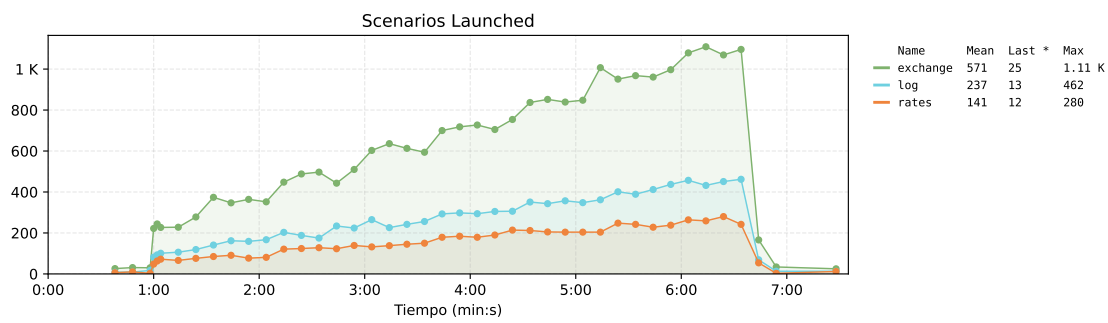


Figura 93. Escenarios de la propuesta 2.

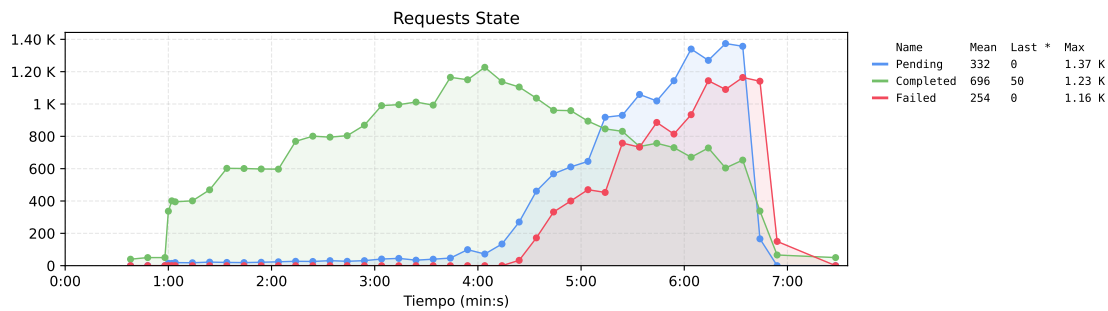


Figura 94. Estado de las requests de la propuesta 2.

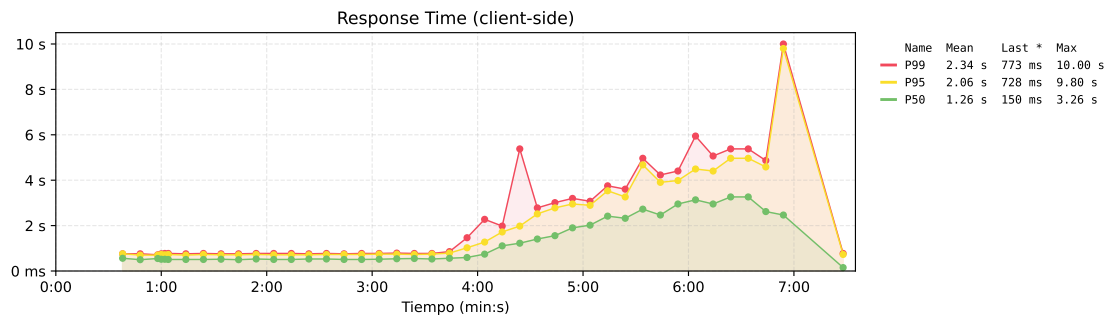


Figura 95. Tiempo de respuesta de la propuesta 2.

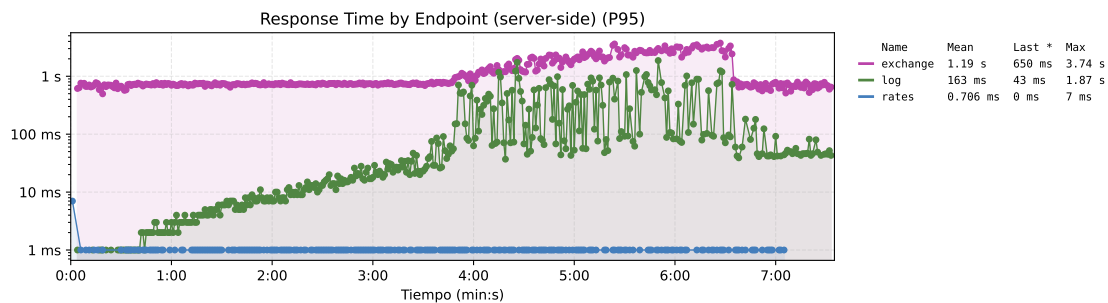


Figura 96. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

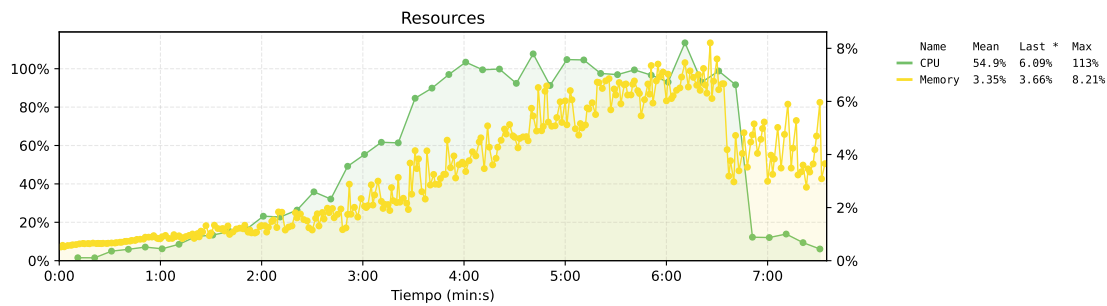


Figura 97. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

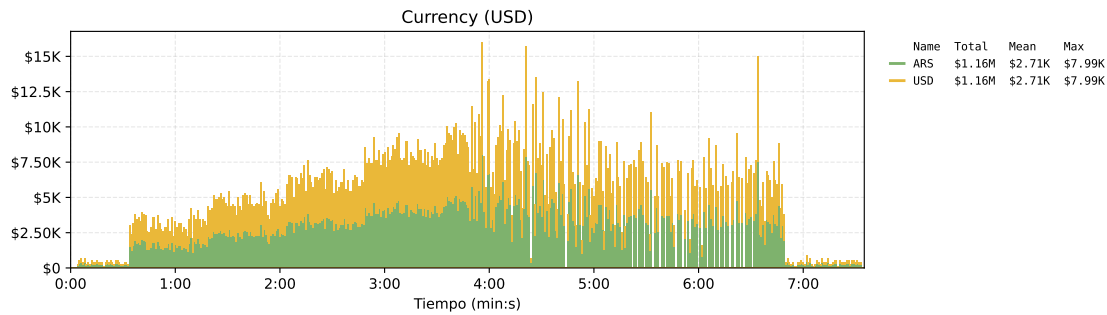


Figura 98. Volumen operado en cada moneda en la propuesta 2.

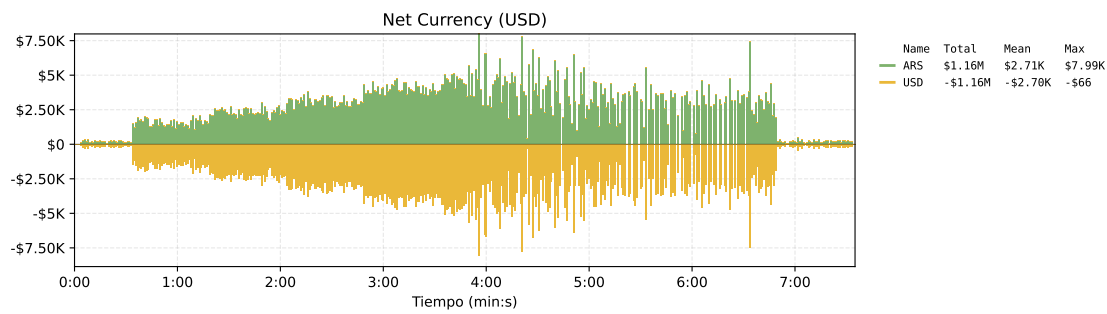


Figura 99. Neto operado en cada moneda en la propuesta 2.

A.10. Resultados completos del escenario de resistencia sobre la propuesta 2

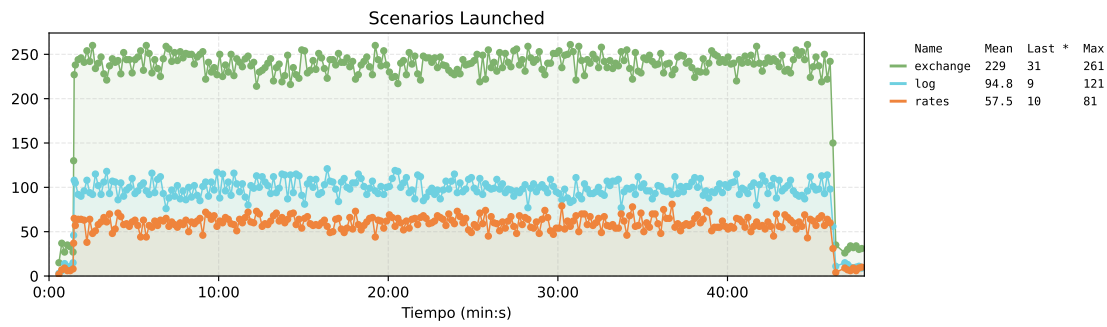


Figura 100. Escenarios de la propuesta 2.

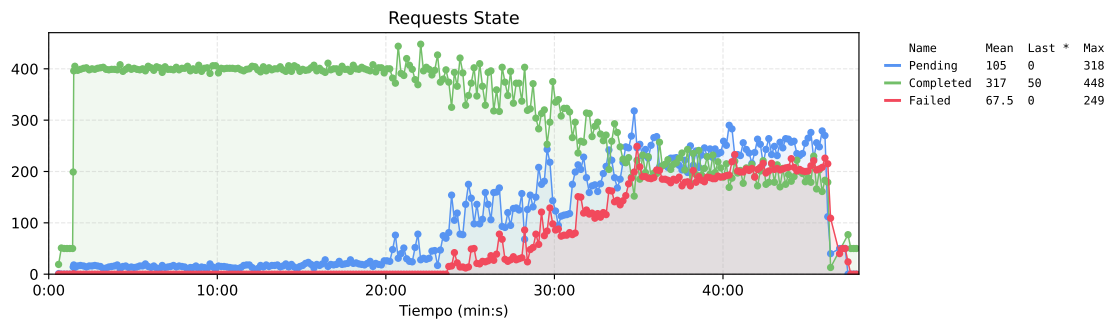


Figura 101. Estado de las requests de la propuesta 2.



Figura 102. Tiempo de respuesta de la propuesta 2.

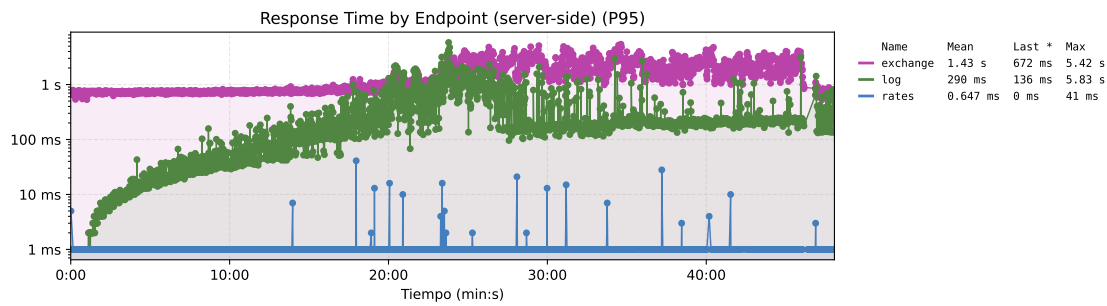


Figura 103. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

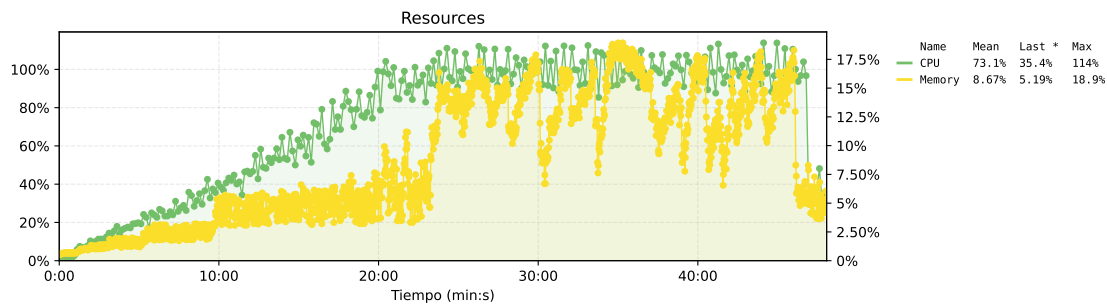


Figura 104. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

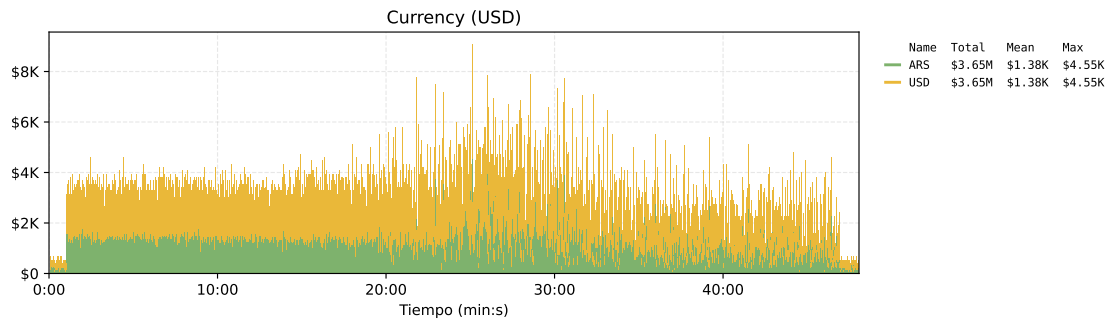


Figura 105. Volumen operado en cada moneda en la propuesta 2.

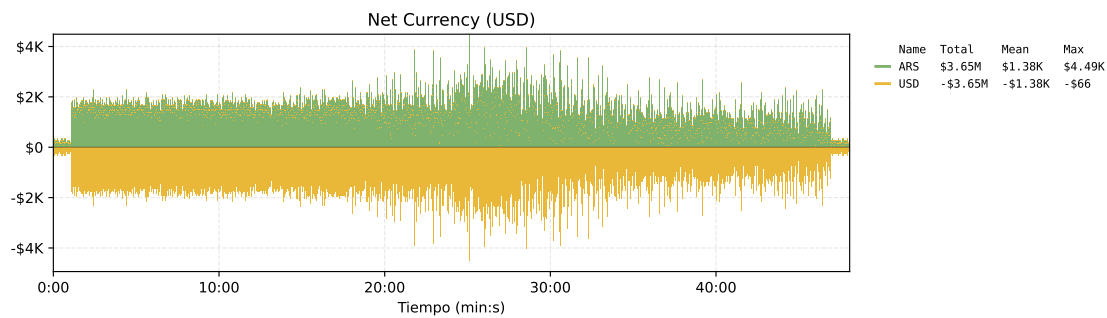


Figura 106. Neto operado en cada moneda en la propuesta 2.

A.11. Resultados completos del escenario de caos sobre la propuesta 2

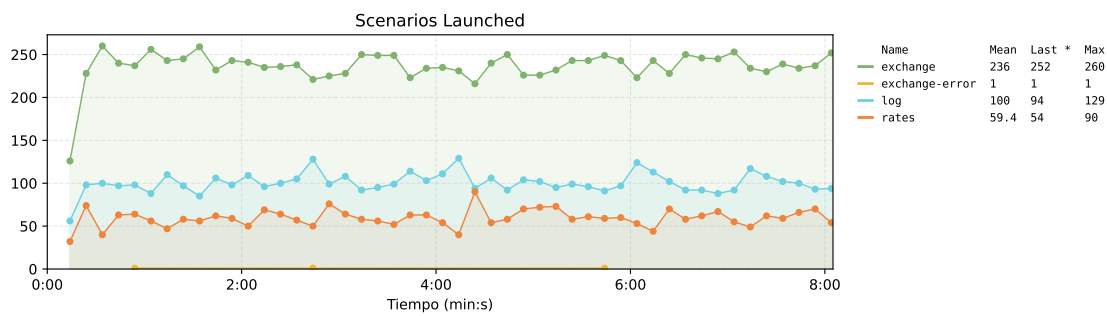


Figura 107. Escenarios de la propuesta 2.

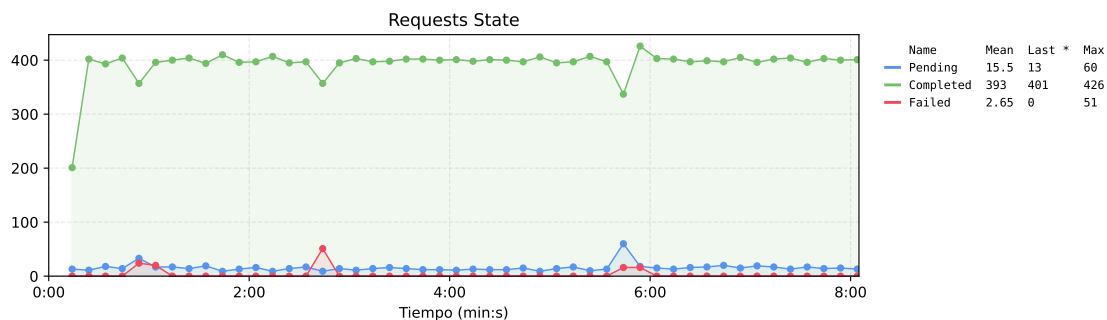


Figura 108. Estado de las requests de la propuesta 2.

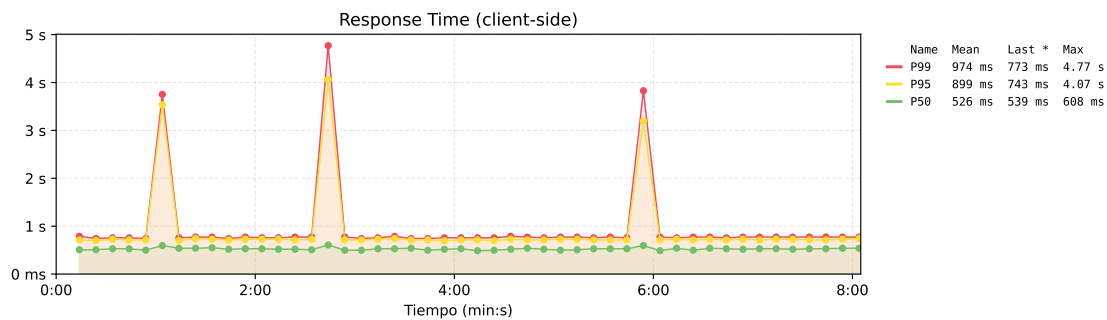


Figura 109. Tiempo de respuesta de la propuesta 2.

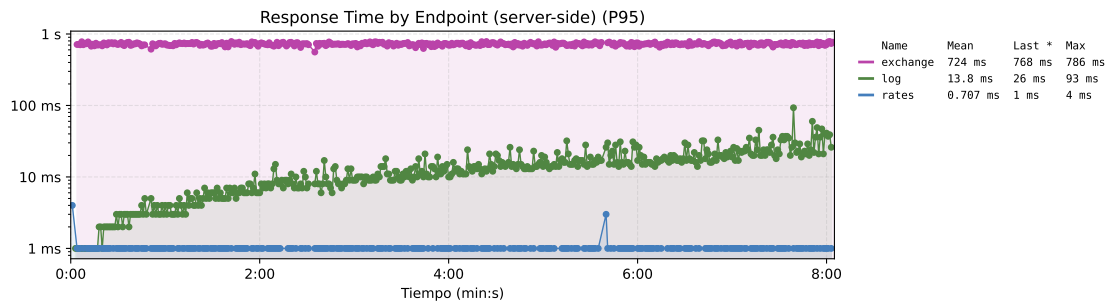


Figura 110. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logaritmica.

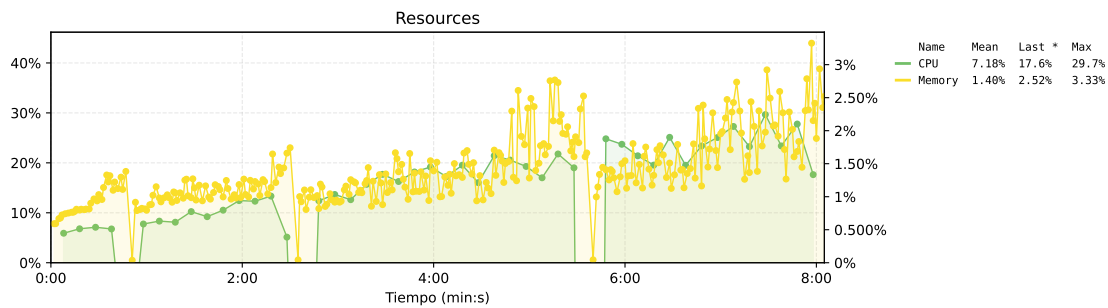


Figura 111. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

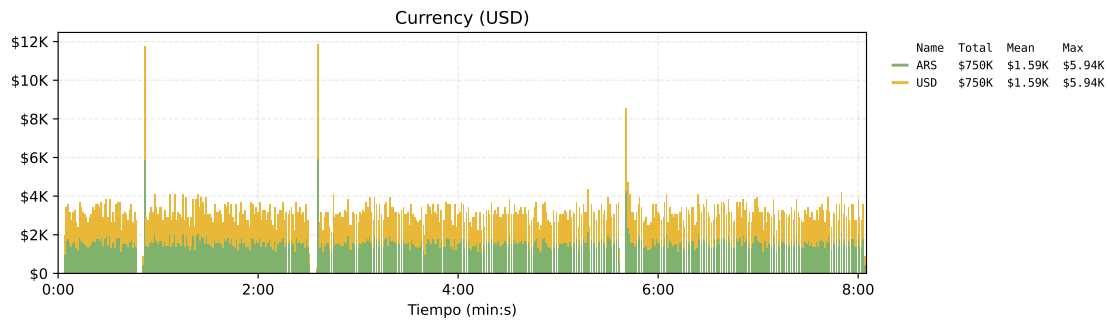


Figura 112. Volumen operado en cada moneda en la propuesta 2.

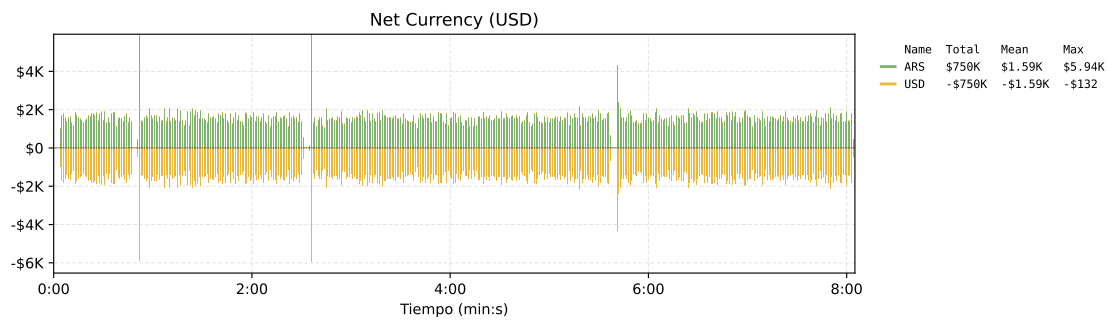


Figura 113. Neto operado en cada moneda en la propuesta 2.